

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Parte 3 – Prof. Luigi Di Caro

Riassunto Completo

Teoria (Capitoli 1–10) e Laboratori

Semantica computazionale · Definizioni · Privacy e misinformation ·
Polisemia ·
Basic level · Pre-LLM NLP · RAG · LLM e prompting · Valutazione ·
Conclusioni

A cura di:

Alessandro Olivero

Matricola 915069

Fonte: Dispense ufficiali del Prof. Luigi Di Caro

Anno Accademico: 2025/2026

Documento di studio personale – non sostituisce le dispense ufficiali

29 giugno 2026

Indice

Come leggere questo documento	2
I Teoria (dalle Dispense Ufficiali)	3
1 Semantica Computazionale: Rappresentazione, Uso e Generazione del Significato	3
1.1 Le Tre Prospettive sul Significato	3
1.2 Dal Significato al Question Answering	4
1.3 Dai Sistemi Modulari ai Modelli Generativi	5
1.4 Ambiguità: WSD e WSI	5
1.4.1 Word Sense Disambiguation (WSD)	6
1.4.2 Word Sense Induction (WSI)	6
1.5 Risorse Lessico-Semantiche: un Confronto	7
1.6 Il Ruolo delle Definizioni	7
2 Definizioni, Semasiologia e Ricerca Onomasiologica	7
2.1 Introduzione	7
2.2 Le Definizioni come Oggetti Linguistici e Computazionali	8
2.3 Tipi di Definizioni	8
2.4 Aspetti Metodologici e Risorse Lessico-Semantiche	9
2.5 Il Problema della Qualità Definizioneale	10
2.6 Valutazione delle Definizioni: Approcci Intrinseci ed Estrinseci	10
2.7 Semasiologia e Onomasiologia	11
2.8 Definizioni, Retrieval e NLP	11
2.9 Esercizio di Laboratorio: Complessità Definizioneale (Lab 1)	11
2.10 Esercizio di Laboratorio: Valutazione delle Definizioni – Content-to-Form (Lab 2)	12
2.11 Esempi Applicativi e Laboratorio (Estensione)	13
2.12 Perché Tutto Questo è Importante per l’NLP Contemporaneo	13
2.13 Conclusione	13
3 Privacy, Rilascio dei Dati e Disinformazione Online	14
3.1 Introduzione	14
3.2 Il Rilascio dei Dati come Problema Semantico	14
3.3 La Stima del Rischio e il Ruolo dell’Utente	14
3.4 Accesso all’Informazione e Misinformation	15
3.5 Explainability e Fiducia nei Sistemi	15
3.6 LLM, RAG e Conoscenza Esterna	15
3.7 Il Legame tra Privacy e Misinformation	15
3.8 Sperimentazione: Annotazione Assistita e Ruolo del RAG	16
3.8.1 Struttura Generale dell’Attività	16
3.8.2 Annotazione PRE e POST: una Prospettiva Comparativa	16
3.8.3 Definizioni Operative	16

3.8.4	Valutazione della Confidenza e della Spiegazione	16
3.8.5	Interpretazione dei Risultati	17
3.9	Discussione: Annotazione, Explainability e User Empowerment	17
3.10	Il Progetto KURAMi: Framework Concettuale e Contributi	18
3.11	Conclusione	18
3.12	Bibliografia Essenziale per Area Tematica	18
4	Polisemia: Complessità, Struttura e Variazioni Multilingue	19
4.1	Introduzione: dalla Disambiguazione alla Complessità	19
4.2	Dal Problema della Disambiguazione al Problema della Complessità	19
4.3	Dimensioni della Polisemia	20
4.3.1	Numero di Sensi	20
4.3.2	Distribuzione dei Sensi	20
4.3.3	Separabilità Semantica	20
4.4	Polisemia e Embeddings Contestuali	21
4.5	Polisemia e Variazione Multilingue	21
4.5.1	Non Esiste una Polisemia Universale	21
4.6	Una Metodologia per Misurare la Complessità della Polisemia	21
4.6.1	Livello 1 – Distribuzione dei Sensi	22
4.6.2	Livello 2 – Separabilità nello Spazio Vettoriale	22
4.6.3	Livello 3 – Visualizzazione in Spazi Ridotti	22
4.7	Polisemia e Qualità dei Corpora	22
4.8	Effetto della Complessità Polisemica sulla Disambiguazione	22
4.9	Polisemia come Struttura Semantica	22
4.10	Polisemia e Large Language Models	23
4.11	Conclusione	23
5	Il Problema della Complessità Lessicale: Basic Level	24
5.1	Introduzione	24
5.2	L'Origine: Roger Brown e la Domanda Fondamentale	24
5.3	Basic Level vs. Middle Level: Collocazione Gerarchica	25
5.4	Definizioni Moderne del Basic Level	25
5.5	Proprietà e Fenomenologia del Basic Level	26
5.6	Applicazioni del Basic Level in NLP	27
5.7	Metriche e Risorse Computazionali	28
5.8	Esperimenti e Annotazione	28
5.9	Sfide Aperte e Direzioni Future	29
5.10	Conclusioni	29
6	Pre-LLM NLP: dal Text Mining in poi	30
6.1	Introduzione: Perché Studiare il Pre-LLM	30
6.2	Una Storia in Quattro Atti: l'Evoluzione del NLP	31
6.3	Dalle Frequenze al Significato: l'Intuizione Distribuzionale	33
6.4	I Task del Text Mining: Cosa Cercavamo di Risolvere	34
6.4.1	Clustering: Scoprire Strutture Nascoste	34
6.4.2	Classificazione: Apprendere da Esempi Etichettati	35
6.4.3	Summarization: l'Essenza del Testo	35
6.4.4	Information Retrieval: dal Boolean Search al Semantic Search	35
6.4.5	Topic Modeling: Scoprire Temi Nascosti	36

6.5	Conclusione: Cosa Abbiamo Imparato, Dove Stiamo Andando	37
6.6	Laboratorio LAB-4: Topic Modeling con BERTopic	39
7	Retrieval-Augmented Generation: tra Memoria Esterna, Retrieval e LLM	40
7.1	Introduzione	40
7.2	Il Problema dell'Aggiornamento	41
7.3	Dal Sapere Implicito al Sapere Accessibile	41
7.4	Information Retrieval: il Problema del Trovare	41
7.4.1	Approcci Classici: TF-IDF e BM25	41
7.5	Semantica Distribuzionale e Retrieval Semantico	42
7.5.1	Modelli per Dense Retrieval	42
7.6	Architettura RAG: uno Schema Concettuale	42
7.7	Chunking: il Problema della Granularità della Conoscenza	43
7.8	Vector Database e Scalabilità	43
7.9	Hybrid Search: Combinazione di Livelli Lessicale e Semantico	44
7.10	RAG come Integrazione di Paradigmi	44
7.11	Evaluation: Misurare la Qualità di un Sistema RAG	44
7.12	Limiti e Questioni Aperte	44
7.13	Direzioni Future	45
7.14	Conclusione	45
7.15	Lab-5 (Dispense): Costruire un Sistema RAG su Articoli Scientifici NLP	45
8	Large Language Models, Prompting e Human-AI Interaction	46
8.1	Introduzione	46
8.2	Dai Modelli Base agli Instruction-Tuned Models	46
8.3	Prompting: l'Arte di Comunicare con gli LLM	47
8.3.1	Principi Fondamentali	47
8.3.2	Strategie Avanzate	47
8.4	Task Comuni con gli LLM	48
8.5	Limiti e Problematiche degli LLM	49
8.6	Human-AI Interaction: la Nuova Frontiera	49
8.6.1	Dal Task-Oriented al Goal-Oriented	49
8.6.2	Nuovi Pattern di Interazione	49
8.6.3	Sfide nella Human-AI Interaction	50
8.6.4	Design Principles per Interfacce LLM	50
8.6.5	Casi d'Uso Emergenti	51
8.7	Conclusioni	51
9	Valutazione per la Terza Parte del Corso di TLN	51
9.1	La Filosofia di Valutazione	52
9.2	Struttura dell'Esame	52
9.3	Parte I: Teoria e Pensiero Critico	52
9.4	Parte II: Laboratorio e Implementazione	52
9.5	Griglia di Valutazione Sintetica	53

10 Conclusioni: Semantica, Conoscenza e Futuro del NLP	53
10.1 Il Percorso Compiuto	54
10.2 Temi Trasversali	54
10.3 Sfide Aperte	55
10.4 Direzioni Future	56
10.5 Riflessioni Metodologiche	56
10.6 Il Futuro è Conversazione	56
 II Laboratori (Dettaglio Implementativo e Risultati)	 56
11 Introduzione: Contenuto vs. Forma Lessicale	57
12 I Quattro Tipi Concettuali	57
13 Similarità Lessicale e Semantica (Lab 1)	58
13.1 SimLex – Indice di Jaccard	58
13.2 SimSem – Coseno TF-IDF	58
13.3 Confronto tra coppie – tutte le combinazioni	59
13.4 Risultati Lab 1	60
14 Content-to-Form: Ricerca Onomasiologica (Lab 2)	60
14.1 Il Task	60
14.2 Procedura Lab 2	60
14.3 Struttura Genus-Differentia	61
14.4 Risultati Lab 2	62
15 Topic Modeling con BERTopic (Lab 4)	62
15.1 Architettura della Pipeline	62
15.2 Stadio 1: Embedding	62
15.3 Stadio 2: UMAP (Riduzione Dimensionale)	63
15.4 Stadio 3: HDBSCAN (Clustering)	63
15.5 Stadio 4: c-TF-IDF (Estrazione Keywords Topic)	63
15.6 Configurazioni Testate	63
15.7 Risultati Comparativi	64
16 RAG: Retrieval-Augmented Generation	64
16.1 Architettura Generale	65
16.2 Retrieval Semantico con FAISS + SciBERT	65
16.3 Retrieval Lessicale con BM25	65
16.4 Hybrid Search: Reciprocal Rank Fusion (RRF)	66
16.5 Componenti del Sistema (Lab 5)	67
17 Metriche di Valutazione (Lab 5)	68
17.1 Precision@K	68
17.2 Context Relevance (CR)	68
17.3 Answer Relevance (AR)	68
17.4 Faithfulness (FF)	69
17.5 Robustness	69

18 Risultati Lab 5: Esperimento Multi-Corpus	69
18.1 Disegno Sperimentale	69
18.2 Tabella Risultati Completa	70
18.3 Analisi Critica dei Risultati	70
19 Confronto Approcci: Tabella Sinottica	71
20 Teoria: Embedding e Retrieval	72
20.1 Dense vs. Sparse Representation	72
20.2 Sentence Transformers	73
20.3 SciBERT vs. BERT generale	73
20.4 Cos'è HDBSCAN: Intuizione	73
21 Riepilogo Formule	74
22 Domande Tipiche da Esame	74

Come leggere questo documento

Il documento usa sei tipi di riquadro colorato per segnalare a colpo d'occhio il tipo di contenuto:

Definizione (blu) – concetti, terminologia e distinzioni teoriche fondamentali.

Formula (verde acqua) – espressioni matematiche e metriche quantitative.

Esempio (azzurro) – casi concreti tratti dalle dispense o dai laboratori.

Risultato (arancione) – tabelle riassuntive e dati sperimentali.

Codice (verde) – snippet Python rilevanti per l'implementazione.

Domanda/riflessione (viola) – spunti critici, citazioni dirette dalle dispense, domande da esame.

Struttura: Parte I (Teoria) segue l'ordine dei dieci capitoli ufficiali delle dispense; Parte II (Laboratori) riporta in dettaglio l'implementazione e i risultati dei laboratori effettivamente svolti, con riferimenti incrociati ai concetti teorici corrispondenti.

Parte I

Teoria (dalle Dispense Ufficiali)

1 Semantica Computazionale: Rappresentazione, Uso e Generazione del Significato

1.1 Le Tre Prospettive sul Significato

La semantica computazionale si può dividere in **tre prospettive complementari** (non alternative): ciascuna risponde a domande diverse sul significato.

1. Semantica lessicale – il significato come inventario. Studia il significato delle parole come entità discrete organizzate in una rete di relazioni semantiche (sinonimia, iponimia, meronimia, antonimia, polisemia). Esempio: *cane* e *gatto* sono iponimi di *animale*; *zampa* è meronimo di *cane*; *bank* è polisemico (banca/riva del fiume).

Risorse tipiche: **WordNet** (Miller, 1995) organizza i concetti in *synset* (insiemi di sinonimi) collegati da relazioni semantiche, oltre 117.000 synset per l'inglese; **BabelNet** (Navigli & Ponzetto, 2012) estende WordNet integrando Wikipedia in oltre 500 lingue; **VerbNet** (Kipper et al., 2008) si concentra su semantica verbale e ruoli tematici.

Problema della granularità (Kilgariff, 1997): come decidere se due usi di una parola costituiscono due sensi diversi o due sfumature dello stesso senso? WordNet distingue 10 sensi per il verbo “run”, ma la distinzione è arbitraria e dipende dalle scelte dei lessicografi.

2. Semantica formale – il significato come verità. Definisce il significato delle espressioni linguistiche in termini di *condizioni di verità*, usando la logica matematica come metalinguaggio. Esempio: “Tutti i gatti sono mammiferi” diventa $\forall x(\text{Gatto}(x) \rightarrow \text{Mammifero}(x))$; “Ogni studente ha letto almeno un libro” diventa $\forall x(\text{Studente}(x) \rightarrow \exists y(\text{Libro}(y) \wedge \text{Letto}(x, y)))$.

Approcci principali: **semantica model-theoretic** (Montague, 1970) – le espressioni linguistiche sono funzioni da modelli a valori di verità; **Discourse Representation Theory** (Kamp, 1981) – gestisce anafora e quantificazione in contesto; **lambda calculus** (Church, 1940) – formalismo per la semantica composizionale.

Limite fondamentale: non sa rappresentare vaghezza, soggettività e concetti graduali (“Questo film è noioso”, “Maria è piuttosto alta”), che richiedono estensioni come la fuzzy logic (Zadeh, 1965).

3. Semantica distribuzionale – il significato come uso. Rompe completamente con le prime due: non parte da definizioni o regole, ma dai dati osservabili dell'uso linguistico.

Ipotesi Distribuzionale. Il significato di una parola è determinato dalla distribuzione dei suoi contesti d'uso in un corpus. Firth (1957): “*You shall know a word by the company it keeps*”. Harris (1954): “parole che occorrono in contesti simili tendono ad avere significati simili”.

Analizzando grandi corpora si costruiscono *embeddings*: rappresentazioni vettoriali in cui la vicinanza tra vettori riflette similarità semantica. **Word2Vec** (Mikolov et al., 2013, CBOW e Skip-gram) predice parole dai contesti o viceversa; **GloVe** (Pennington et al., 2014) fattorizza matrici di co-occorrenza globale; **BERT/ELMo** (Devlin et al., 2019) produce embeddings contestuali che cambiano in base al contesto (risolvendo il problema della polisemia).

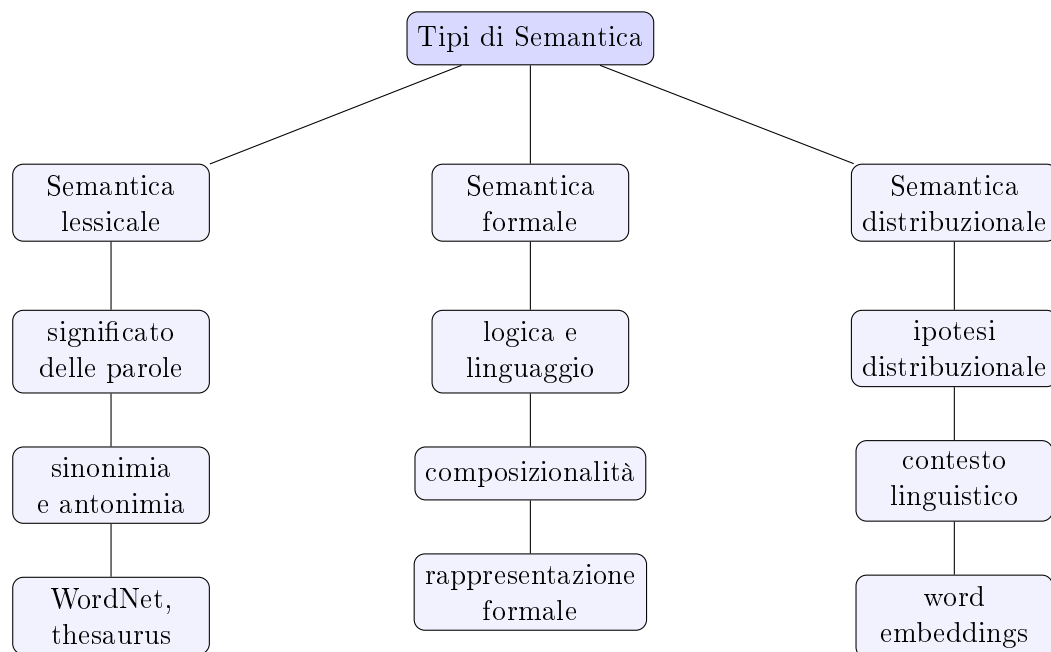
Esempio algebrico famoso: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$ – le relazioni semantiche sono catturate geometricamente nello spazio vettoriale.

Similarità coseno (misura standard di similarità tra vettori):

$$\text{sim}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot |\vec{b}|} = \cos(\theta)$$

Valori vicini a 1 = alta similarità; vicini a 0 = ortogonalità (non correlazione); vicini a -1 = opposizione.

Problema dell'interpretabilità: il significato non è più esplicito ma distribuito in centinaia di dimensioni. Cosa significa la dimensione 247 in un embedding di 300 dimensioni? Non c'è risposta chiara; alcune dimensioni catturano feature semantiche (animato/inanimato), la maggior parte resta opaca.



1.2 Dal Significato al Question Answering

Il **Question Answering (QA)** è uno dei problemi storici del NLP: leggere un testo e rispondere a domande su di esso. Sistemi storici: BASEBALL (Green et al., 1961), LUNAR (Woods, 1973).

Non tutte le domande sono uguali. Esistono diverse categorie cognitive:

- **Fattuale**: “Quando è nata Marie Curie?” → recupero di fatti (IR)
- **Definitoria**: “Cos’è un algoritmo?” → semantica lessicale
- **Causale**: “Perché il cielo è blu?” → ragionamento inferenziale
- **Ipotetica/controfattuale**: “Cosa succederebbe se...?” → simulazione mentale

Tassonomia di Li & Roth (2002) per la classificazione delle domande: ABBR (abbreviation, “Cosa significa NASA?”), ENTY (entity, “Cos’è un transistor?”), DESC (description, “Come funziona un motore?”), HUM (human, “Chi ha scritto 1984?”), LOC (location), NUM (numeric).

Analisi multi-livello di “Chi ha dipinto la Gioconda?”: livello **lessicale** (“Gioconda” = opera d’arte), livello **sintattico** (soggetto=X, predicato=dipingere, oggetto=Gioconda), livello **semantico** (relazione CREATOR-OF(X, Gioconda)), livello **pragmatico** (tipo di risposta attesa = HUMAN).

1.3 Dai Sistemi Modulari ai Modelli Generativi

Storicamente il QA era affrontato con **pipeline modulari**: (1) classificazione della domanda, (2) NER, (3) relation extraction, (4) WSD, (5) retrieval, (6) answer extraction/generation. Sistemi come **Watson** di IBM (Ferrucci et al., 2010, vincitore di Jeopardy! nel 2011) usavano oltre 100 componenti combinando approcci statistici, simbolici e knowledge-based.

Con i **Large Language Model** (GPT-3/4, Claude, LLaMA) il paradigma è cambiato: i modelli non si limitano a recuperare informazioni, le **generano** – sintetizzano da fonti diverse, spiegano in modo adattivo, riformulano per livelli di competenza diversi, ragionano (apparentemente) su catene di inferenze.

Generazione vs. Retrieval: alla domanda “Spiega la relatività a un bambino di 8 anni”, un sistema retrieval-based cercherebbe documenti con “relatività”+“spiegazione semplice” e restituirebbe un passaggio estratto. Un LLM generativo produce una spiegazione *originale*, adattata al livello richiesto, con analogie creative.

Nuove criticità introdotte dai generativi: **allucinazioni** (Maynez et al., 2020; Ji et al., 2023) – generazione di fatti plausibili ma falsi; **difficoltà di controllo** – non è chiaro su quali informazioni si basi la risposta; **assenza di grounding** – nessuna garanzia di verità o tracciabilità delle fonti; **bias** – amplificazione di pregiudizi nei dati di training (Bender et al., 2021). Il **RAG** (Lewis et al., 2020) tenta di combinare i due paradigmi: prima recuperare, poi generare basandosi sui documenti recuperati.

1.4 Ambiguità: WSD e WSI

Ambiguità Lessicale. Una parola è ambigua quando può riferirsi a concetti diversi in contesti diversi. **Polisemia**: sensi correlati (“libro” = oggetto fisico vs. contenuto testuale). **Omonimia**: sensi non correlati (“banco” = pesce vs. mobile).

1.4.1 Word Sense Disambiguation (WSD)

WSD assume un inventario predefinito di sensi $S = \{s_1, \dots, s_n\}$ (tipicamente da WordNet) per una parola w in contesto C , e consiste nel trovare:

$$s^* = \arg \max_{s \in S} P(s | w, C)$$

“The bank can guarantee deposits will eventually cover future tuition costs”. *bank* ha 4 sensi in WordNet (istituto finanziario, riva di fiume, riserva, banca del sangue). Il contesto (“guarantee”, “deposits”, “costs”) indica il senso #1 (istituto finanziario).

Approcci: **knowledge-based** (Lesk, 1986 – overlap tra contesto e definizioni); **supervised** (Zhong & Ng, 2010 – classificazione con feature contestuali); **neural** (Raganato et al., 2017 – embeddings contestuali).

Limiti intrinseci: eccessiva specificità dei sensi (WordNet distingue sensi che gli umani non percepiscono come diversi); copertura incompleta (nuovi sensi emergono continuamente, es. “viral” nel senso di contenuto virale online); soggettività (accordo inter-annotatore spesso basso, 70-80%); domain specificity (sensi specialistici mancanti).

1.4.2 Word Sense Induction (WSI)

WSI scopre automaticamente i sensi di una parola raggruppando i suoi contesti d'uso in cluster semanticamente coerenti, **senza** un inventario di sensi predefinito (a differenza di WSD).

Per il verbo *play*: Cluster 1 (musica): “play guitar”, “play piano”; Cluster 2 (sport): “play football”, “play tennis”; Cluster 3 (gaming): “play videogames”; Cluster 4 (teatro): “play a role”, “play Hamlet”. Ogni gruppo è un senso indotto automaticamente.

Algoritmi: **context clustering** (Schütze, 1998 – clustering di vettori contestuali); **graph-based** (Agirre et al., 2006 – grafi di co-occorrenza e community detection); **topic modeling** (Brody & Lapata, 2009 – LDA per sensi come topic latenti); **neural clustering** (Amrami & Goldberg, 2019).

Vantaggi: adattabilità a domini specifici, nessuna dipendenza da risorse esterne, scopre sensi emergenti. Svantaggi: numero di sensi non determinato a priori, valutazione difficile (non esiste gold standard), sensi spesso non interpretabili.

Valutazione: metodo Pseudo-word. Gale et al. 1992; Schütze 1992. Si fondono artificialmente due parole non correlate (es. *apple* + *keyboard* → *apple-keyboard*), si sostituiscono tutte le occorrenze originali con la pseudo-word, e si verifica se il sistema riassegna correttamente ogni istanza al senso originale (clustering). Limiti: i sensi creati non riflettono ambiguità reale; i sensi artificiali sono più distanti di quelli naturali; infla artificialmente le performance. Valutazioni più realistiche usano dataset annotati come i SemEval tasks (Agirre et al., 2007, 2010, 2013).

1.5 Risorse Lessico-Semantiche: un Confronto

Risorsa	Potere espr.	Scalabilità	Sorgente	Ambig./Sogg.
WordNet/BabelNet	Alto	Bassa	Manuale	Bassa
Property norms	Alto	Bassa	Esperimenti/Crowd	Alta
FrameNet	Alto	Media	Annotatori esperti	Media
Common-sense (ConceptNet)	Molto alto	Alta	Crowd-sourcing/ML	Molto alta
Visual Attributes	Medio	Medio-alta	Immagini annotate	Medio-alta
Word/sense embeddings	Molto alto	Molto alta	Testi larga scala	Alta
Corpus manager	Variabile	Alta	Corpora reali	Media

Altre risorse: **FrameNet** (Baker et al., 1998) organizza il lessico in “frame semantici” (es. frame COMMERCIAL_TRANSACTION: buyer, seller, goods, money), utile per semantic role labeling. **PropBank** (Palmer et al., 2005) annota ruoli semantici verbali, focus su alternanze di diathesis (voce attiva/passiva). **ConceptNet** (Speer et al., 2017) conoscenza di senso comune crowd-sourced (es. (coffee, UsedFor, staying awake)). **Property Norms** (McRae et al., 2005) proprietà percettive/funzionali da dati psicologici sperimentali (es. “banana”: is yellow, is curved). **Tendenza recente**: integrare risorse simboliche con embeddings (retrofitting, Faruqui et al., 2015; knowledge graph embeddings, Bordes et al., 2013).

1.6 Il Ruolo delle Definizioni

Tutte le risorse condividono un principio: **descrivere un concetto tramite altri concetti**. Questo, noto come **circularità definitoria**, è inevitabile ma problematico: “Cane” → mammifero → animale → essere vivente → ... forma un grafo concettuale potenzialmente ciclico. Dove si ferma la regressione? Esistono “primitive semantiche” (Wierzbicka, 1996)?

Tipi di definizione: per **genere e differenza specifica** (Aristotele); **ostensiva** (per esempi); **operazionale** (per procedure). Quali proprietà sono rilevanti: necessarie vs. sufficienti (teoria classica dei concetti); **proprietà prototipiche** (Rosch, 1973 – un pettirosso è più “uccello” di un pinguino); proprietà distintive. Come distinguere concetti simili: “cup” vs “mug” vs “glass” hanno confini sfumati dipendenti dal contesto culturale; **minimal pairs** differiscono per una sola feature critica.

Prospettiva computazionale moderna: gli LLM imparano definizioni implicitamente dai pattern d’uso, senza rappresentazioni esplicite. Un modello che usa correttamente “cane” in migliaia di contesti “comprende” cosa è un cane? È una variante del **Chinese Room argument** (Searle, 1980): performance linguistica $\equiv$ comprensione semantica?

2 Definizioni, Semasiologia e Ricerca Onomasiologica

2.1 Introduzione

La semantica computazionale non coincide con un unico modo di rappresentare il significato, ma con una pluralità di approcci (lessicale, formale, distribuzionale, generativo)

che condividono una domanda di fondo: *come si descrive un concetto?* Ogni volta che definiamo una parola compiamo un'operazione teorica e pratica delicata: selezioniamo alcune proprietà, ne escludiamo altre, decidiamo cosa è centrale e cosa periferico, scegliamo un livello di generalità. **La definizione non è solo uno strumento lessicografico: è una forma di modellizzazione del significato.**

Una definizione è, in termini generali, un testo progettato per guidare il lettore – o, più in generale, il *consumer* – verso un possibile significato associato a un termine in un determinato contesto. La relazione tra termini e significati *non è univoca*: un singolo termine può essere associato a una molteplicità di significati, e uno stesso significato può essere espresso tramite definizioni differenti, più o meno accurate, più o meno orientate a uno scopo comunicativo specifico.

Il problema della definizione è quindi anche un problema di: selezione delle proprietà rilevanti; granularità concettuale; controllo dell'ambiguità; adeguatezza rispetto al pubblico di riferimento; rappresentabilità computazionale.

2.2 Le Definizioni come Oggetti Linguistici e Computazionali

Definire significa molto più che spiegare una parola: significa stabilire un ponte tra una forma linguistica e una struttura concettuale. Le definizioni svolgono quindi una funzione insieme: **descrittiva** (forniscono informazioni sul significato); **distintiva** (separano un concetto da altri concetti simili); **comunicativa** (devono essere comprensibili a un destinatario); **operativa** (possono essere utilizzate in compiti di retrieval, classificazione, disambiguazione e generazione). La definizione **non è mai neutra**: cambia in funzione dello scopo, del dominio, del livello di specializzazione e persino del tipo di utente previsto.

Il termine *microscopio* può essere definito in modi diversi, mettendo in rilievo aspetti diversi pur riferendosi allo stesso concetto:

- **genus-differentia**: “strumento ottico usato per osservare oggetti troppo piccoli per essere visti a occhio nudo”;
- **per esempio d'uso**: “apparecchio usato in laboratorio per ingrandire cellule, tessuti o microrganismi”;
- **operazionale**: “dispositivo che consente l'osservazione ad alta risoluzione di strutture microscopiche tramite lenti o altri sistemi di ingrandimento”.

2.3 Tipi di Definizioni

Esistono diversi tipi di definizioni, classificabili in base alla strategia esplicativa adottata; nessuna tipologia è universalmente migliore delle altre – la scelta dipende dal compito e dall'obiettivo.

- **Genus-differentia** (modello classico aristotelico): identifica una categoria generale (*genus*) e poi specifica le proprietà distintive (*differentia*) che separano il concetto dai suoi vicini semantici. Es.: *Cane*: mammifero domestico della famiglia dei canidi; *Triangolo*: poligono con tre lati. Vantaggi: esplicita la collocazione tassonomica, facilita la classificazione, si presta bene alla formalizzazione computazionale. Limite: non sempre sufficiente per concetti astratti, prototipici o culturalmente variabili.

- **Definizioni per esempio:** mostrano casi tipici o contesti d'uso invece di esplicitare la struttura concettuale. Es.: *Euristica*: una strategia mentale come “provare la soluzione più semplice per prima” o “seguire una regola approssimativa”. Più accessibili per un pubblico non specialista, ma meno rigorose dal punto di vista classificatorio.
- **Definizioni per riferimento o rinvio:** descrivono un concetto richiamandone altri già noti. Es.: *Felino*: animale appartenente alla stessa famiglia del gatto, del leone e della tigre. Utili in sistemi lessicali fortemente interconnessi, ma rischiano di amplificare la circolarità definitoria.
- **Definizioni per parafrasi, sinonimia o descrizione operativa:** riformulano il significato in modo più accessibile o funzionale. Es.: *Pericolo*: situazione in cui esiste la possibilità di un danno; *Algoritmo*: insieme ordinato di passi per risolvere un problema. Frequenti nei contesti educativi e nella comunicazione tecnico-divulgativa.

2.4 Aspetti Metodologici e Risorse Lessico-Semantiche

Tutte le strategie definitorie si basano sulla descrizione di una parola tramite l'uso di altre parole. Questo porta a riflessioni fondamentali: come si descrive un concetto? Quali caratteristiche devono essere menzionate? Quali proprietà lo distinguono da altri concetti? Come si scrive una definizione, e chi decide se è “giusta”? Come si valuta la qualità e la completezza di una definizione? Queste domande sono centrali per chi costruisce dizionari, thesauri, knowledge graph, risorse terminologiche, dataset per il NLP, o sistemi generativi che devono produrre spiegazioni coerenti.

Altre dimensioni rilevanti per le risorse: **espressività** (quanto una risorsa rappresenta conoscenze ricche, astratte e relazionali); **scalabilità** (quanto può essere ampliata automaticamente o semi-automaticamente); **sorgente** (annotatori esperti, crowd-sourcing, dati testuali/visivi, esperimenti cognitivi); **ambiguità e soggettività** (alcune risorse tentano di eliminare l'ambiguità, altre la accolgono come parte costitutiva del linguaggio naturale).

Risorsa	Potere espr.	Scalabilità	Sorgente	Ambig./Sogg.
Dizionari elettronici	Medio-alto	Medio-bassa	Manuale	Bassa
Property norms	Alto	Bassa	Esperimenti/Crowd	Alta
Frames	Alto	Media	Annotatori esperti	Media
Common-sense knowledge	Molto alto	Alta	Crowd-sourcing/ML	Molto alta
Visual Attributes	Medio	Medio-alta	Immagini annotate	Medio-alta
Word/sense embeddings	Molto alto	Molto alta	Testi su larga scala	Alta
Corpus manager	Variabile	Alta	Corpora reali	Media

Esempi concreti: **dizionari elettronici** (WordNet, BabelNet) organizzano il lessico tramite synset, glosse e relazioni semantiche; le **Property Norms** raccolgono proprietà associate ai concetti tramite esperimenti cognitivi (utili per studiare quali caratteristiche vengono spontaneamente considerate salienti); il **common-sense knowledge** (es. ConceptNet) rappresenta relazioni come *UsedFor*, *CapableOf*, *AtLocation*; i **Visual Attributes** raccolgono proprietà fisionomiche/percettive (colore, forma, texture); i **corpus manager** (es. SketchEngine) permettono di esplorare grandi corpora e osservare usi reali, collocazioni e pattern definitori.

2.5 Il Problema della Qualità Definizione

Una definizione non è utile solo perché è grammaticalmente ben formata: deve essere anche informativa, discriminante e adeguata a uno scopo. **Criteri di qualità:** **chiarezza** (è comprensibile senza ulteriori spiegazioni?); **accuratezza** (descrive correttamente il concetto?); **specificità** (distingue il concetto da altri simili?); **completezza** (include le proprietà essenziali?); **adeguatezza** (è appropriata per il pubblico/task?); **disambiguazione** (evita letture alternative fuorvianti?).

Per il concetto *pericolo*: Definizione A – “qualcosa di brutto che può succedere” (intuitiva ma vaga); Definizione B – “situazione o condizione in cui esiste la possibilità di un danno, di una perdita o di una minaccia” (più strutturata, informativa, adatta a un contesto lessicografico/computazionale).

La qualità definizionale dipende anche dal **tipo di concetto**: i concetti **concreti** tendono a essere più facilmente ancorabili a proprietà percettive e funzionali; i concetti **astratti** richiedono spesso relazioni, situazioni, effetti o inferenze; i concetti **specifici** sono spesso più vincolati; i concetti **generici** tendono ad avere confini più sfumati.

2.6 Valutazione delle Definizioni: Approcci Intrinseci ed Estrinseci

Valutazione intrinseca: misura la qualità considerando esclusivamente le proprietà interne della definizione, senza riferimento a un task applicativo – verifica quanto sia coerente, informativa e semanticamente allineata al concetto. Metriche: **similitudine semantica** (cosine similarity tra embedding della definizione e del termine); **overlap lessicale** (Jaccard similarity, overlap coefficient); **ricchezza lessicale** (varietà/distribuzione del vocabolario); **coerenza interna** (consistenza tra le proprietà espresse).

Valutazione estrinseca: misura la qualità in base al contributo che la definizione fornisce alla performance di un sistema in un task applicativo reale (la definizione non è valutata isolatamente, ma come componente di un sistema più ampio). Task tipici: WSD (es. algoritmo di Lesk), Question Answering, Information Retrieval, Machine Translation, Text Classification.

Aspetto	Intrinseca	Estrinseca
Velocità	Alta	Bassa
Costo computazionale	Basso	Alto
Necessità di task	No	Sì
Utilità reale	Limitata	Elevata
Dipendenza dal modello	Alta	Media
Generalizzabilità	Bassa	Media

I due approcci rispondono a esigenze diverse e **non sono alternativi in senso stretto**. **Limiti della valutazione intrinseca:** forte dipendenza dal modello usato (es. BERT vs. GPT), incapacità di valutare la qualità lessicografica in senso umano, sensibilità al contesto. **Limiti della valutazione estrinseca:** dipendenza dal task specifico, influenza del dataset e dei suoi bias, error propagation da altri moduli del sistema.

Best practice – triangolazione delle valutazioni: la strategia più efficace combina più modalità di valutazione: l'intrinseca è utile per uno screening rapido e lo sviluppo iterativo; l'estrinseca è fondamentale per la validazione finale; l'analisi qualitativa permette di interpretare i risultati e comprendere le discrepanze. Quando le diverse valutazioni convergono si ha maggiore fiducia nella qualità della definizione; le divergenze possono segnalare problemi strutturali da approfondire.

2.7 Semasiologia e Onomasiologia

Approccio semasiologico (forma → contenuto): parte da un termine linguistico e mira a determinarne il significato/i significati possibili – l'approccio tipico dei dizionari tradizionali (es. data la parola *penna*, il dizionario fornisce definizioni come “strumento per scrivere”, “piuma di uccello”). Centrale in: definizione lessicale, disambiguazione di sensi, consultazione di dizionari, interpretazione di termini ambigui.

Approccio onomasiologico (contenuto → forma): procede nel verso opposto – si parte da un concetto/definizione/descrizione e si cerca di risalire alla forma linguistica che può esprimerlo (es. data “strumento usato per scrivere con inchiostro”, si risale a *penna*). Problema cruciale non solo in lessicografia, ma anche in NLP, information retrieval, text generation e supporto alla scrittura.

Aspetti correlati alla ricerca onomasiologica: **dizionari analogici** (organizzati per concetti/campi semantici, non per forma alfabetica); **tip-of-the-tongue phenomenon** (il parlante dispone del contenuto concettuale ma non riesce a recuperare la forma lessicale appropriata); il **meccanismo genus-differentia** può facilitare la risalita al concetto corretto. Molti sistemi computazionali moderni combinano continuamente i due movimenti: comprendono parole/frasi per risalire a concetti, e generano parole/frasi a partire da rappresentazioni concettuali o istruzioni.

2.8 Definizioni, Retrieval e NLP

Nei sistemi classici una definizione ben costruita poteva essere usata per classificare concetti, popolare risorse lessicali, facilitare la disambiguazione, guidare la ricerca in basi di conoscenza. Nei sistemi moderni (LLM, approcci retrieval-based) le definizioni hanno assunto funzioni aggiuntive: diventano **query semantiche** per il recupero di concetti; sono usate come **prompt** o descrizioni di target concettuali; possono essere confrontate automaticamente tramite similarità lessicale/semantica; fungono da ponte tra linguaggio naturale e strutture formali (synset, frame, entità in un knowledge graph). Una definizione può quindi essere vista come una **rappresentazione linguistica compressa** di un contenuto semantico.

2.9 Esercizio di Laboratorio: Complessità Definizione (Lab 1)

LAB-1 – Schema delle Dispense. Misurazione dell'overlap lessicale tra definizioni per concetti generici/specifici e concreti/astratti, su **4 concetti**: due concreti (uno generico, es. *pantalone*; uno specifico, es. *microscopio*) e due astratti (uno generico, es. *pericolo*; uno specifico, es. *euristica*).

Schema in 5 fasi: (1) **creazione condivisa** delle definizioni per i 4 concetti, scegliendo liberamente la strategia (genus-differentia, esempi, parafrasi, o combinazioni); (2) **analisi collettiva** delle scelte linguistiche, del grado di specificità, delle difficoltà per i concetti astratti rispetto ai concreti; (3) **task di analisi lessicale e semantica**: calcolo del grado di sovrapposizione lessicale e semantica tra le definizioni dello stesso concetto prodotte da persone diverse; (4) **calcolo della similarità lessicale (SimLex)** – conteggio dei termini in comune dopo stopword removal/lemmatizzazione, oppure Jaccard/overlap coefficient – e **semantica (SimSem)** – rappresentazione vettoriale (Word2Vec, GloVe, fastText, BERT, Sentence-BERT) + cosine similarity – aggregando i valori per concretezza e specificità; (5) **riflessione**: la sovrapposizione lessicale è risultata più alta o più bassa delle aspettative? La sovrapposizione semantica riflette meglio l'intuizione di somiglianza? Ci sono differenze tra concetti concreti/astratti o generici/specifici?

Implementazione concreta del laboratorio (concetti usati: Albero/Teiera/Musica/Etica, vedi dettagli e codice in Sez. 13.1 e seguenti): **SimLex** (Jaccard su insiemi di parole lemmatizzate) misura overlap lessicale puro; **SimSem** (coseno TF-IDF) misura prossimità semantica. Risultato chiave: per concetti astratti SimSem > SimLex (il modello vettoriale allinea sinonimi che l'overlap puro non vede); per i concetti concreti i punteggi sono più alti e più vicini tra loro perché il referente fisico vincola fortemente il vocabolario usato da persone diverse.

2.10 Esercizio di Laboratorio: Valutazione delle Definizioni – Content-to-Form (Lab 2)

LAB-2 – Ricerca Onomasiologica. I dizionari comuni partono dalla forma per arrivare al contenuto. I dizionari **analogici** funzionano al contrario: non si ricerca per parola, ma per definizione o contenuto concettuale (**ricerca onomasiologica**). **Obiettivo:** verificare se e quanto le definizioni prodotte nel Lab 1 siano sufficientemente informative e ben strutturate da permettere l'identificazione del *synset* corretto in WordNet (o risorsa equivalente).

Procedimento: (1) per ciascun concetto, raccogliere tutte le definizioni disponibili; (2) analizzare ogni definizione (singolarmente o aggregata) cercando in WordNet il synset che meglio corrisponde al significato espresso; (3) durante la ricerca: attenzione alle ambiguità lessicali, uso di iperonimi e relazioni tassonomiche come guida, principio del *genus* per orientarsi nel grafo semantico; (4) registrare, per ogni definizione, il synset individuato e motivare la scelta.

Domande guida: Quanto è stato facile risalire al synset corretto usando solo la definizione? Quali definizioni si sono rivelate più efficaci? Alcune definizioni hanno generato ambiguità o errori di interpretazione, perché? Le definizioni basate su genus-differentia si sono dimostrate più utili di altre?

Implementazione concreta del laboratorio (vedi dettagli, codice e risultati completi in Sez. 14.1): procedura automatizzata con TF-IDF + coseno tra definizione e 5 glosse candidate per concetto, predizione = synset con score massimo. **Risultato chiave:** i concetti concreti sono fino a $2.1\times$ più recuperabili degli astratti (Accuracy: Albero 89.3% vs. Musica 32.1%) – il referente fisico vincola il vocabolario rendendo le definizioni più standardizzate e quindi più facili da far corrispondere alla glossa corretta tramite TF-IDF. Limite: TF-IDF fallisce sulla sinonimia pura (richiede embedding contestuali per i concetti astratti).

2.11 Esempi Applicativi e Laboratorio (Estensione)

Come esercizio di consolidamento (singolarmente o a gruppi): (1) **integrare risorse eterogenee** (es. WordNet + ConceptNet, o FrameNet + embeddings) per valutare la complementarità tra definizioni esplicite, relazioni simboliche e similarità distribuzionale; (2) **estendere/arricchire una risorsa esistente** (es. migliorare la copertura semantica delle Property Norms, aggiungere metadati sulle strategie definitorie). Criteri di valutazione: corretta comprensione delle risorse, completezza dell'analisi, capacità critica/interpretativa, qualità delle visualizzazioni (tabelle, grafi, heatmap), codice ben documentato, riflessione finale su utilità e limiti.

2.12 Perché Tutto Questo è Importante per l'NLP Contemporaneo

A prima vista, il problema delle definizioni potrebbe sembrare marginale rispetto all'NLP contemporaneo (embeddings, transformer, modelli generativi). In realtà è vero il contrario: ogni sistema che deve spiegare un concetto, distinguere tra significati simili, recuperare un termine da una descrizione, generare una risposta definitoria, o valutare la correttezza semantica di un testo, si confronta direttamente col problema della definizione. Anche i LLM, pur non possedendo definizioni esplicite nel senso tradizionale, mostrano comportamenti che dipendono fortemente da regolarità definizionali apprese dai dati – quando un modello completa correttamente “Uno strumento usato per osservare oggetti molto piccoli si chiama ___” sta sfruttando associazioni distribuzionali, pattern definitori e conoscenza lessicale interiorizzata durante l'addestramento. Questo non significa che il modello “posseda” il significato come una risorsa simbolica, ma mostra che il confine tra definizione esplicita e conoscenza implicita è oggi uno dei punti più interessanti della riflessione semantica.

2.13 Conclusione

Punti chiave del capitolo: una definizione non è solo una spiegazione, ma una forma di rappresentazione del significato; i diversi tipi di definizione mettono in luce strategie descrittive differenti; la qualità definizionale dipende da chiarezza, specificità, adeguatezza e capacità di disambiguazione; l'approccio **semasiologico** parte dalla forma per arrivare al contenuto, l'**onomasiologico** parte dal contenuto per arrivare alla forma; questi problemi hanno applicazioni dirette in lessicografia, retrieval semantico, classificazione e NLP.

Il punto più importante è forse un altro: **definire significa sempre scegliere una prospettiva sul concetto**. Non esiste quasi mai una definizione perfettamente neutrale, definitiva o completa – esistono definizioni più o meno utili, più o meno efficaci, più o meno adatte a un certo scopo cognitivo o computazionale. Studiare le definizioni non significa occuparsi di un residuo teorico della lessicografia tradizionale, ma affrontare una domanda centrale della semantica: *in che modo il significato può essere reso accessibile, confrontabile e operativamente utilizzabile?*

3 Privacy, Rilascio dei Dati e Disinformazione Online

3.1 Introduzione

Nel NLP contemporaneo il linguaggio non è solo un oggetto di analisi, ma un punto di intersezione tra sistemi computazionali, pratiche sociali e processi decisionali. All'interno di questo ecosistema emergono due problemi profondamente interconnessi pur essendo spesso trattati separatamente: il **rilascio di dati personali e sensibili** e la **circolazione di contenuti non verificati o fuorvianti**. Da un lato gli utenti condividono informazioni che possono esporli a rischi di privacy, talvolta senza esserne consapevoli; dall'altro sono immersi in flussi informativi in cui la distinzione tra informazione affidabile e misinformazione non è sempre immediata.

Il punto centrale non è soltanto *proteggere* ciò che viene comunicato o *filtrare* ciò che viene ricevuto, ma costruire sistemi che aiutino l'utente a comprendere entrambi i processi. La nozione di **user empowerment** è chiave: l'obiettivo non è sostituire il giudizio umano con decisioni automatiche, ma fornire strumenti che lo rendano più informato, consapevole e controllabile.

3.2 Il Rilascio dei Dati come Problema Semantico

Il rilascio dei dati è spesso interpretato come una questione puramente tecnica, legata alla presenza o assenza di determinati attributi identificativi – ma questa visione è riduttiva. Nei contesti digitali contemporanei, il rischio di un contenuto **non dipende solo da ciò che è esplicitamente dichiarato, ma anche da ciò che può essere inferito**. Un testo apparentemente innocuo può contenere elementi che, combinati con altre informazioni disponibili online, permettono di ricostruire aspetti rilevanti dell'identità o del comportamento di un individuo.

Il problema della privacy è quindi **intrinsecamente semantico**: riguarda la relazione tra espressioni linguistiche, contesto e interpretazione. Non basta individuare entità sensibili: bisogna capire come tali entità si inseriscano in una rete di relazioni e quali inferenze possano essere attivate. Il rischio privacy emerge da un'interazione tra *contenuto*, *contesto* e *conoscenza esterna*. Questo supera i modelli binari (visibile/nascosto): è più corretto adottare strategie di **rilascio controllato**, dove il contenuto viene trasformato, generalizzato o parzialmente oscurato per ridurre il rischio mantenendo utilità informativa.

3.3 La Stima del Rischio e il Ruolo dell'Utente

Il rischio privacy non è una proprietà statica ma dipende da molteplici fattori: serve quindi un modello *dinamico* che consideri la presenza di dati sensibili, il contesto di pubblicazione, le possibilità di propagazione e la combinabilità con altre fonti. In questo quadro l'utente assume un ruolo centrale: non si tratta più di applicare regole predefinite, ma di **supportare decisioni situate**. Un sistema efficace dovrebbe fornire indicazioni comprensibili sul rischio associato a un contenuto, esplicitare quali elementi contribuiscono a tale rischio, e suggerire possibili strategie di mitigazione.

Questo capitolo fa riferimento agli strumenti sviluppati nell'ambito del progetto nazionale PRIN **KURAMi**, che propone un approccio integrato ai problemi del rilascio dei dati personali e della disinformazione online, utilizzando i concetti di *privacy release* e *misinformation*.

3.4 Accesso all'Informazione e Misinformation

Se il rilascio dei dati riguarda ciò che l'utente **espone**, l'accesso all'informazione riguarda ciò a cui l'utente è **esposto**. Qui il problema non è solo recuperare contenuti pertinenti, ma garantire che siano affidabili. Tradizionalmente i sistemi di Information Retrieval sono progettati per massimizzare la *rilevanza* rispetto a una query – ma rilevanza non coincide con veridicità: un documento può essere altamente pertinente e al tempo stesso inaccurato o fuorviante. La misinformation introduce quindi una nuova dimensione di valutazione: non solo se un documento risponde alla query, ma se ne fornisce una rappresentazione adeguata dei fatti – richiedendo modelli che integrino coerenza con fonti affidabili, presenza di evidenze e qualità della fonte.

3.5 Explainability e Fiducia nei Sistemi

Un elemento comune a privacy e misinformation è la necessità di rendere **interpretabili** le decisioni dei sistemi: quando un sistema segnala un rischio o valuta un contenuto come poco affidabile, l'utente deve poter comprendere il perché. L'explainability non si limita a rendere il sistema più trasparente, ma contribuisce a costruire un rapporto di fiducia basato sulla comprensione – senza spiegazioni, le decisioni automatiche rischiano di apparire arbitrarie o di essere accettate in modo acritico.

3.6 LLM, RAG e Conoscenza Esterna

I LLM hanno ampliato le possibilità di analisi e generazione del linguaggio, ma hanno anche sollevato questioni sull'affidabilità delle informazioni prodotte. Il problema principale sono le **hallucinations**: generazione di contenuti plausibili ma non supportati da evidenze, particolarmente critica quando i modelli vengono usati per verificare informazioni o supportare decisioni. Per affrontarlo si diffonde l'uso di tecniche di **Retrieval-Augmented Generation**, che ancorano la generazione a fonti esterne, migliorando la verificabilità dei contenuti.

3.7 Il Legame tra Privacy e Misinformation

Un aspetto interessante è l'ipotesi di un legame tra rilascio dei dati personali e **vulnerabilità alla misinformation**: questa relazione non è immediata, ma suggerisce che determinati comportamenti di disclosure possano essere associati a maggiore esposizione a contenuti non verificati. Se esiste un legame tra ciò che l'utente condivide e ciò a cui è esposto, privacy e qualità dell'informazione **non possono essere trattate come problemi indipendenti**, ma diventano due aspetti di una stessa dinamica informativa.

3.8 Sperimentazione: Annotazione Assistita e Ruolo del RAG

3.8.1 Struttura Generale dell'Attività

L'esercitazione si articola in **tre fasi distinte**, da svolgere in ordine preciso:

1. **Fase 1 (PRE)**: ciascun partecipante annota una serie di contenuti basandosi *esclusivamente* sul testo fornito. Non è consentito consultare l'output del sistema RAG, né (per la privacy) usare fonti esterne. Per la misinformation è ammessa solo una consultazione minima di fonti, limitata alla verifica di fatti, senza usare modelli generativi.
2. **Fase 2**: viene reso visibile l'output del sistema RAG (classificazione + spiegazione). L'output è solo letto e interpretato, **non** modificato.
3. **Fase 3 (POST)**: i partecipanti rivedono la propria annotazione alla luce dell'informazione fornita dal sistema – possono mantenerla, modificarla, o confermarla con un diverso livello di confidenza. Devono inoltre valutare la qualità della spiegazione (chiarezza, pertinenza, completezza).

3.8.2 Annotazione PRE e POST: una Prospettiva Comparativa

La distinzione PRE/POST è il cuore metodologico dell'esperimento. L'annotazione **PRE** è un giudizio non assistito, basato solo su comprensione individuale e conoscenze pregresse. L'annotazione **POST** introduce una dimensione riflessiva: il partecipante confronta la propria interpretazione con quella del sistema. Può: (a) portare a una **revisione** della classificazione; (b) **rafforzare** una valutazione già formulata, aumentando la confidenza; (c) risultare **poco convincente**, evidenziando limiti/ambiguità nella spiegazione. L'obiettivo *non* è stabilire qual è la risposta "corretta", ma analizzare il processo di costruzione del giudizio e come viene influenzato da un supporto computazionale.

3.8.3 Definizioni Operative

- **Privacy** (annotazione **binaria**, violazione sì/no): si considera violazione la presenza di informazioni sensibili che potrebbero identificare individui, esporre credenziali o dati finanziari, rivelare comunicazioni riservate, o rendere inferibili aspetti personali come abitudini, relazioni o condizioni di salute.
- **Misinformation** (scala **ordinale**): distingue contenuti accurati, parzialmente fuorvianti, misti e prevalentemente falsi. La scala graduata riconosce che la misinformation non è un fenomeno binario, ma si distribuisce lungo un continuum.

3.8.4 Valutazione della Confidenza e della Spiegazione

Per ogni annotazione si richiede un livello di **confidenza**, sia in fase PRE che POST – elemento fondamentale per distinguere cambiamenti nella decisione da cambiamenti nella sicurezza percepita. Si valuta inoltre la qualità della spiegazione RAG su tre dimensioni: **chiarezza** (quanto è comprensibile), **pertinenza** (quanto è rilevante rispetto al contenuto), **completezza** (quanto copre gli aspetti importanti del caso). Si richiede infine di

indicare esplicitamente *se e come* il sistema ha influenzato la valutazione – per distinguere cambiamenti nella classificazione, variazioni nella confidenza, o effetti più sottili nei commenti qualitativi.

3.8.5 Interpretazione dei Risultati

I dati raccolti permettono di osservare in quali casi il sistema ha avuto impatto maggiore (sulla classificazione o sulla confidenza), e se esistono differenze tra il dominio privacy e quello misinformation. Spiegazioni chiare, pertinenti e complete sono più probabile che influenzino il giudizio; spiegazioni vaghe tendono a essere ignorate o a generare incertezza.

Esempio di schema di annotazione PRE–RAG–POST.

ID	Testo (estratto)	PRE	Conf.	RAG	POST	Conf.
P5	“Joe, we would like to make an urgent request... Michelle Lokay’s hotline number 18000...”	Si	4	Violation Si		5
P8	“Glad you finally made it!... Saturday I have a haircut...”	No	3	Violation Si		4

Interpretazione: in P5 la violazione è esplicita e facilmente identificabile già in fase preliminare (nomi propri, numeri interni, contatti telefonici) – il RAG **rafforza** la valutazione, aumentando la confidenza. In P8 la violazione è meno evidente perché basata su inferenze contestuali (informazioni implicite su relazioni familiari e routine) – il sistema rende espliciti elementi non inizialmente considerati, **portando a una revisione** della classificazione (da No a Violation Si). Questo mostra che il supporto del sistema non si limita a confermare decisioni già prese, ma può rendere visibili forme di rischio meno immediate, legate a informazioni implicite, routine e relazioni personali.

Estensione dello schema: nel dataset completo ogni istanza include anche: motivazione dell’annotazione nella fase PRE; valutazione dettagliata della spiegazione RAG; indicazione esplicita dell’influenza del sistema sulla decisione; commenti qualitativi dell’annotatore.

3.9 Discussione: Annotazione, Explainability e User Empowerment

L’attività collega in modo concreto tre dimensioni: la valutazione del contenuto, la spiegabilità del sistema, il ruolo attivo dell’utente. Emerge chiaramente che il giudizio non è un processo statico, ma una costruzione dinamica influenzabile da nuove informazioni e da strumenti di supporto. Nel quadro dello user empowerment, **un sistema non contribuisce all’empowerment semplicemente fornendo una risposta**, ma nella misura in cui rende l’utente capace di comprendere, valutare e, se necessario, *contestare* tale risposta – l’explainability è quindi una **condizione** per un’interazione equilibrata tra umano e macchina, non un accessorio. L’influenza del sistema non è uniforme: in alcuni

casi corregge errori evidenti, in altri introduce dubbi o interpretazioni alternative, in altri ancora risulta poco convincente – la qualità delle spiegazioni, la competenza dell'utente e il contesto determinano questa variabilità.

3.10 Il Progetto KURAMi: Framework Concettuale e Contributi

KURAMi (*Knowledge-based, explainable User empowerment in Releasing private data and Assessing Misinformation in online environments*) è un progetto nazionale **PRIN**, finanziato dal Ministero dell'Università e della Ricerca, coordinato dall'Università degli Studi di Torino (Prof. Di Caro) in collaborazione con l'Università di Milano-Bicocca e l'Università Statale di Milano.

Si basa su **due pilastri**:

1. **Rappresentazione della conoscenza**: costruzione di modelli che catturino non soltanto informazioni fattuali, ma anche relazioni argomentative, fonti e gradi di certezza.
2. **User empowerment**: progettazione di sistemi che non sostituiscono il giudizio umano, ma lo supportano tramite spiegazioni, visualizzazioni e strumenti di controllo.

Sito ufficiale: <https://kurami.disco.unimib.it> (pubblicazioni, dataset annotati, documentazione, demo interattive).

3.11 Conclusione

La privacy non si riduce a un controllo dell'accesso, ma va interpretata come un problema di **inferenza e contesto**; la misinformation richiede modelli che integrino rilevanza e veridicità; explainability e knowledge grounding sono essenziali per sistemi affidabili. Privacy e misinformation convergono verso una visione dell'NLP come disciplina non solo tecnica, ma anche **epistemica e sociale**: il linguaggio non è solo un dato da elaborare, ma un mezzo attraverso cui si costruiscono conoscenza, identità e decisioni. Progettare sistemi che operano sul linguaggio significa quindi intervenire direttamente su questi processi, con tutte le responsabilità che ne derivano.

3.12 Bibliografia Essenziale per Area Tematica

Privacy e anonimizzazione: Sweeney (2002) *k-anonymity: A model for protecting privacy* – il modello classico di k-anonymity (già visto nel corso di Privacy); Dwork (2006) *Differential privacy*; Lison et al. (2021) *Anonymisation models for text data*; Pilán et al. (2022) *TAB: Text Anonymization Benchmark*.

Misinformation e fact-checking: Thorne et al. (2018) *FEVER: a large-scale dataset for fact extraction and verification*; Zhou & Zafarani (2020) *A survey of fake news*; Kotonya & Toni (2020) *Explainable automated fact-checking for public health claims*.

Explainability: Ribeiro et al. (2016) *"Why should I trust you?" – LIME*; Doshi-Velez & Kim (2017) *Towards a rigorous science of interpretable ML*; Jacovi & Goldberg (2020) *faithfully interpretable NLP systems*; Atanasova et al. (2020) *Generating fact checking explanations*.

Retrieval-Augmented Generation: Lewis et al. (2020) *RAG for knowledge-intensive NLP tasks*; Guu et al. (2020) *REALM*; Gao et al. (2023) *RARR: researching and revising what LMs say*.

Human-AI interaction e user empowerment: Shneiderman (2020) *Human-centered AI*; Bansal et al. (2021) *Does the whole exceed its parts? Effect of AI explanations on team performance*.

Risorse online: KURAMi Project (<https://kurami.disco.unimib.it>); ACL Anthology (<https://aclanthology.org>); Papers With Code (<https://paperswithcode.com>).

4 Polisemia: Complessità, Struttura e Variazioni Multilingue

4.1 Introduzione: dalla Disambiguazione alla Complessità

Nelle lezioni precedenti l'ambiguità lessicale era stata affrontata in termini di **disambiguazione**: dato un contesto, quale senso è quello corretto (WSD)? In questo capitolo si cambia punto di vista: non *come scegliere* il senso corretto, ma:

Quanto è complesso il sistema dei sensi di una parola? Come si distribuiscono questi sensi nell'uso reale? Quanto sono separabili tra loro? E come varia tutto questo tra lingue diverse?

La polisemia **non è un fenomeno binario**: alcune parole hanno pochi sensi e un significato dominante, altre presentano molti sensi distribuiti in modo relativamente equilibrato; alcune hanno sensi fortemente distinti, altre mostrano confini semantici sfumati. Possiamo quindi considerare la polisemia come una **proprietà graduale, strutturata e misurabile** – importante perché la complessità polisemica influenza direttamente la difficoltà di task come WSD, Machine Translation, Information Retrieval, QA, e l'uso di embeddings contestuali e LLM.

4.2 Dal Problema della Disambiguazione al Problema della Complessità

La prospettiva tradizionale: nel paradigma classico della WSD il problema si esprime come $(w, C) \rightarrow s_i$, dove w è una parola, C il contesto, s_i il senso corretto da un inventario. Questa formulazione lascia in ombra domande importanti: perché alcune parole sono molto più difficili da disambiguare di altre? Il numero di sensi è sufficiente a spiegare la difficoltà? Conta di più il numero dei sensi o la loro distribuzione? I sensi sono sempre chiaramente separabili?

La polisemia come complessità intrinseca: la ricerca recente suggerisce che la difficoltà della disambiguazione non dipende solo dai modelli o dai dataset, ma da una proprietà più profonda – **la complessità della polisemia è una proprietà intrinseca della parola e del suo comportamento nell’uso**. Alcune parole sono più complesse non semplicemente perché hanno “molti sensi”, ma perché i loro sensi: sono distribuiti in modo più equilibrato; sono meno facilmente separabili nei contesti reali; presentano relazioni semantiche interne articolate; variano in modo diverso da lingua a lingua.

4.3 Dimensioni della Polisemia

4.3.1 Numero di Sensi

La misura più intuitiva: $N(w) = |\{s_1, s_2, \dots, s_n\}|$. Esempi: *oxygen* o *giraffe* hanno profilo quasi monosemico; *bank* o *paper* hanno pochi sensi ma non banali; *run*, *charge*, *set* mostrano polisemia molto elevata. **Limite:** questa misura è insufficiente – una parola può avere molti sensi nel dizionario ma usarne abitualmente solo pochi, e le risorse lessicali dipendono dalle scelte di granularità (WordNet distingue sensi che gli umani non percepiscono come nettamente separati).

4.3.2 Distribuzione dei Sensi

Due parole con lo stesso numero di sensi possono avere complessità molto diversa: Parola A con due sensi distribuiti 95%/5% è meno complessa di Parola B con distribuzione 50%/50% (deve scegliere tra due interpretazioni *entrambe* plausibili).

Entropia dei sensi:

$$H(w) = - \sum_i p(s_i|w) \log p(s_i|w)$$

dove $p(s_i|w)$ è la probabilità del senso s_i dato il lemma w . **Interpretazione:** entropia bassa → un senso dominante; entropia alta → sensi più bilanciati; maggiore entropia → maggiore complessità polisemica.

school: nella maggior parte dei casi significa “istituzione educativa”, il senso “gruppo di pesci” è raro → entropia bassa. *bank*: in inglese i sensi “istituto finanziario” e “riva del fiume” possono essere entrambi rilevanti → entropia più alta. *run*: numerosi sensi con distribuzione articolata → alta complessità.

4.3.3 Separabilità Semantica

Non tutti i sensi sono uguali: alcuni sono semanticamente lontani, altri vicini e sfumati, altri ancora collegati da relazioni regolari (metafora o metonimia). Esempi: *bass* “pesce” vs. *bass* “strumento musicale” sono sensi molto distanti; *paper* “materiale”/“documento”/“articolo” forma una catena di estensioni collegate; *book* indica sia l’oggetto fisico sia il contenuto informativo. **La polisemia non riguarda solo quanti sensi esistono, ma anche quanto sono vicini o lontani tra loro nello spazio semantico.**

4.4 Polisemia e Embeddings Contestuali

Dagli embeddings statici agli embeddings contestuali: nei modelli classici (Word2Vec, GloVe) ogni parola ha un solo vettore – tutti i sensi vengono “compressi” in una singola rappresentazione. Con i modelli contestuali (ELMo, BERT, transformer più recenti), ogni occorrenza genera un embedding diverso a seconda del contesto: $V(w) = \{v_1, v_2, \dots, v_n\}$, dove ogni v_i è la rappresentazione contestuale di un’occorrenza. Questo rende possibile osservare la polisemia come **distribuzione geometrica** di punti nello spazio vettoriale.

Cluster di senso: se i contesti che esprimono sensi diversi sono sufficientemente distinti, i vettori tendono a formare cluster separati (sensi ben distinti → cluster separati; sensi correlati → cluster vicini; sensi sfumati → forte sovrapposizione). Esempi: *bass* tende a produrre cluster lontani; *bank* mostra cluster distinguibili ma non sempre del tutto separati; *paper* dà una struttura più continua e sovrapposta.

Distanza inter-senso (tra i centroidi dei cluster di sensi s_i, s_j):

$$d(s_i, s_j) = 1 - \cos(\vec{s}_i, \vec{s}_j)$$

Distanza alta → sensi ben separati; distanza bassa → sensi simili o parzialmente sovrapposti.

Overlap contestuale: anche se due sensi sono separabili *in media*, possono presentare molti casi borderline – la polisemia va osservata non solo in termini di distanza globale, ma anche di **confusione locale** tra occorrenze reali.

4.5 Polisemia e Variazione Multilingue

4.5.1 Non Esiste una Polisemia Universale

Una stessa area semantica può essere lessicalizzata in modi diversi tra lingue. Esempio classico: inglese *bank* (unica forma) vs. italiano *banca/riva* e spagnolo *banco/orilla* (forme separate).

Asimmetrie cross-linguistiche: la polisemia non è solo una proprietà della parola, ma anche del modo in cui una lingua organizza il proprio lessico. Lingue diverse possono fondere più significati in una sola parola, oppure distribuire significati diversi su parole distinte; possono avere profili polisemici simili solo parzialmente; possono presentare convergenze o divergenze influenzate da vicinanza genealogica, tipologica e culturale.

Implicazioni per il NLP multilingue: queste asimmetrie hanno effetti diretti su machine translation, cross-lingual transfer, allineamento di embeddings multilingui, rappresentazione dei sensi in risorse come BabelNet o WordNet multilingue. Un modello addestrato su una lingua può trovarsi in difficoltà quando incontra, in un’altra lingua, un’organizzazione polisemica diversa da quella appresa.

4.6 Una Metodologia per Misurare la Complessità della Polisemia

Si distinguono **tre livelli di analisi**:

4.6.1 Livello 1 – Distribuzione dei Sensi

Considera la distribuzione empirica dei sensi in corpora annotati. Metriche: numero di sensi attestati; entropia normalizzata; quota del senso più frequente (*top-1 share*). Permettono di distinguere parole quasi monosemiche, con un senso dominante, con distribuzione equilibrata, o ad alta complessità polisemica.

4.6.2 Livello 2 – Separabilità nello Spazio Vettoriale

Osserva come i sensi si organizzano nello spazio degli embeddings contestuali. Metriche: distanza media tra sensi; overlap contestuale; coesione dei cluster; dispersione intra-senso. Permette di distinguere parole con sensi molti ma facilmente separabili, sensi pochi ma molto confondibili, strutture polisemiche compatte o diffuse/complesse.

4.6.3 Livello 3 – Visualizzazione in Spazi Ridotti

Usa tecniche come UMAP o t-SNE per proiettare gli embeddings in due dimensioni. Non sostituisce l'analisi quantitativa, ma la rende intuitivamente interpretabile: cluster compatti → struttura semantica stabile; cluster separati → buona distinguibilità dei sensi; cluster sovrapposti → polisemia più sfumata; strutture diverse tra lingue → differenze nell'organizzazione lessicale.

4.7 Polisemia e Qualità dei Corpora

Distribuzione sbilanciata della complessità: molti corpora usati per WSD e analisi semantica mostrano un problema ricorrente: contengono molte parole a bassa entropia, sottorappresentano i casi realmente complessi, privilegiano parole frequenti e relativamente stabili. Conseguenza: **le performance dei sistemi possono essere sovrastimate** se il dataset contiene soprattutto parole facili.

Valutazione più realistica: è utile costruire benchmark più bilanciati, che includano parole a bassa, media e alta entropia; parole con diversa separabilità geometrica; casi sia monolingui sia multilingui; parole con senso dominante e parole con sensi distribuiti uniformemente.

4.8 Effetto della Complessità Polisemica sulla Disambiguazione

Le metriche di complessità non sono solo descrittive, ma anche **predittive**. Confrontando un sistema classico (es. Lesk) su parole a bassa e alta complessità: le parole a bassa complessità ottengono accuratezza sensibilmente più alta; le parole ad alta complessità sono molto più difficili da trattare; la difficoltà non dipende solo dal numero di sensi, ma dalla **combinazione** di entropia, separabilità e struttura contestuale. La polisemia ha quindi un impatto operativo diretto sulle performance dei sistemi NLP, non solo teorico.

4.9 Polisemia come Struttura Semantica

Un errore frequente è pensare alla polisemia come a un problema o un rumore da eliminare. In realtà è anche una **risorsa cognitiva e linguistica fondamentale**: non è un difetto del lessico, ma una forma di organizzazione del significato. Le estensioni polisemiche seguono spesso principi regolari: **metafora**, **metonimia**, **generalizzazione**, **specializzazione**, convenzioni culturali e discorsive. Esempi: *head* (testa, capo, cima); *paper* (materiale, documento, articolo); *book* (oggetto fisico e contenuto). Questi sensi non sono arbitrari: riflettono modi sistematici in cui i concetti si estendono e si riorganizzano.

4.10 Polisemia e Large Language Models

I modelli generativi attuali non rappresentano esplicitamente i sensi come nodi discreti, ma apprendono regolarità d'uso distribuite su enormi quantità di dati. Questo permette loro di usare spesso correttamente parole polisemiche, adattare il significato al contesto, produrre parafrasi o spiegazioni coerenti. Restano problemi aperti: difficoltà con sensi rari; difficoltà con sensi molto simili; instabilità nelle disambiguazioni sottili; scarsa trasparenza su come il senso venga effettivamente rappresentato internamente. La quantificazione della complessità polisemica può quindi diventare anche uno strumento per valutare i limiti dei modelli neurali contemporanei.

4.11 Conclusione

Il capitolo sposta l'attenzione da una domanda classica (*quale senso è corretto?*) a una più profonda: *com'è organizzata la polisemia di una parola, e quanto è complessa?* Punti chiave: la polisemia è un fenomeno multidimensionale; il numero di sensi non basta a descriverla; la distribuzione dei sensi è cruciale e si misura con l'entropia; la separabilità nello spazio degli embeddings fornisce informazione complementare; la polisemia varia tra lingue e non è universale; influenza direttamente le performance NLP; tecniche di visualizzazione e metriche quantitative permettono di studiarla in modo sistematico. **Studiare la polisemia significa osservare il punto in cui il significato smette di apparire come una lista discreta di etichette e comincia a rivelarsi come uno spazio continuo, dinamico e differenziato tra lingue, usi e contesti.**

Domande di riflessione dalle dispense: Una parola con molti sensi è sempre più complessa di una con pochi sensi? In che modo l'entropia offre una misura più informativa del semplice conteggio? Qual è la differenza tra avere molti sensi e avere sensi molto sovrapposti? La polisemia va rappresentata meglio come inventario discreto o come struttura continua? Perché la polisemia varia tra lingue diverse – dipende solo dal lessico o anche dalla cultura e dall'uso? I LLM rappresentano davvero i sensi o apprendono soltanto configurazioni contestuali?

Bibliografia essenziale: Kilgariff (1997) *I don't believe in word senses*; Pustejovsky (1995) *The Generative Lexicon*; Apresjan (1974) *Regular polysemy*; Navigli (2009) *Word sense disambiguation: a survey*; McCarthy et al. (2004) *Finding predominant word senses in untagged text*; Pasini & Navigli (2020) *XL-WSD*; Miller (1995) *WordNet*; Navigli & Ponzetto (2012) *BabelNet*; Bond & Foster (2013) *Linking and extending an open multilingual wordnet*.

5 Il Problema della Complessità Lessicale: Basic Level

5.1 Introduzione

La ricerca NLP si è tradizionalmente concentrata sull'analisi sintattica/grammaticale, relegando lo studio del vocabolario a un ruolo secondario. Ma **il contenuto semantico è incapsulato nelle parole, che possono avere livelli di complessità radicalmente diversi**: non tutte le parole sono create uguali.

Due descrizioni della stessa scena: **Versione 1 (base)**: “Un *cane* marrone corre nel *parco*.” **Versione 2 (avanzata)**: “Un *canide domestico* dalla *pelliccia fulva* si muove rapidamente attraverso un’*area verde ricreativa urbana*.” Stessa scena, ma la prima usa parole di **base** (*cane*, *parco*, *correre*), la seconda termini **avanzati** – una differenza non meramente stilistica, che riflette distinzioni cognitive, acquisizionali e comunicative profonde.

Perché il basic level è importante: (1) **acquisizione linguistica** – i bambini apprendono prima le parole di base (*cane*) e solo dopo iperonimi (*animale*) e iponimi (*barboncino*); (2) **apprendimento L2** – gli studenti di lingue straniere hanno bisogno di un vocabolario di *sopravvivenza* per comunicare efficacemente; (3) **accessibilità comunicativa** – testi semplificati per persone con disturbi del linguaggio o bassa alfabetizzazione richiedono un vocabolario controllato; (4) **sistemi NLP** – Machine Translation, Text Simplification, Summarization, Question Answering beneficiano tutti di una comprensione esplicita della complessità lessicale.

Il dinamismo del vocabolario: a differenza della grammatica (relativamente stabile), il vocabolario è in costante evoluzione – **neologismi** (*selfie*, *clickbait*, *ghosting*), **shift semantici** (*virale* da biologico a social media), **specializzazione** (termini tecnici diventano mainstream: *cloud*, *streaming*, *podcast*), **variazione sociolinguistica** (comunità diverse usano vocabolari diversi). Questa dinamicità rende ancora più cruciale identificare un *nucleo stabile* di parole fondamentali.

5.2 L’Origine: Roger Brown e la Domanda Fondamentale

“How Shall a Thing be Called?” (1958). Lo psicologo Roger Brown pone la domanda: *data un’entità del mondo reale, quale termine linguistico dovremmo usare per riferirci ad essa?* Di fronte a un’immagine mentale di un cane Golden Retriever, potremmo chiamarlo: *Animale* (iperonimo, troppo generico), *Mammifero* (iperonimo, categoria tassonomica), ***Cane*** (livello base – scelta naturale), *Golden Retriever* (iponimo, troppo specifico per uso quotidiano), *Rex* (nome proprio, massima specificità). **Osservazione empirica di Brown**: in contesti neutri, parlanti nativi convergono sistematicamente verso il livello base (*cane*), evitando spontaneamente sia iperonimi troppo vaghi sia iponimi eccessivamente specifici.

Criteri di Brown per il Basic Level: (1) **brevità** – parole corte, tipicamente 1-2 sillabe (*cane* vs *canide domestico*; *casa* vs *abitazione residenziale*); (2) **concretezza**

– riferimento a entità percepibili sensorialmente (*mela* vs *malus domestica*; *tavolo* vs *mobile d'arredo orizzontale*); (3) **facilità di pronuncia** – fonologia semplice, assenza di cluster consonantici complessi (*gatto* vs *felino domestico*); (4) **alta frequenza** – uso frequente nella lingua parlata e scritta (le parole di base sono tipicamente nel top 5-10% per frequenza).

Il contributo di Rosch: Basic Level Categories (1970s): Eleanor Rosch estende il lavoro di Brown dal punto di vista cognitivo. Le categorie di livello base sono il livello di astrazione al quale si **massimizza la distintività percettiva tra categorie**, si **massimizza la similarità intra-categoria**, e si **minimizza lo sforzo cognitivo** di categorizzazione.

Esperimento empirico di Rosch (mostrando immagini di mobili): livello **superordinato** (*mobile*) – riconoscimento lento, pochi attributi distintivi; livello **base** (*sedia*) – riconoscimento rapido, molti attributi distintivi; livello **subordinato** (*sedia da ufficio ergonomica*) – riconoscimento lento, attributi ridondanti. Risultato: i partecipanti categorizzano più velocemente e accuratamente al livello base.

5.3 Basic Level vs. Middle Level: Collocazione Gerarchica

Le parole si organizzano naturalmente in gerarchie tassonomiche, es. nel dominio degli animali: Essere vivente → Animale (superordinato) → Mammifero (superordinato) → **Cane (LIVELLO BASE)** → Bassotto (subordinato) → Bassotto a pelo lungo (subordinato specifico). Il livello base si colloca tipicamente *a metà* della gerarchia, da cui il termine alternativo **middle level**.

Sequenza di acquisizione (studi sullo sviluppo linguistico infantile): **Fase 1 (12-24 mesi)** – acquisizione del livello base (“cane” per tutti i cani, “palla” per oggetti sferici, “auto” per veicoli); **Fase 2 (24-36 mesi)** – espansione verso iperonimi (apprendimento di “animale” come categoria superiore, comprensione che cane/gatto/uccello sono tutti animali); **Fase 3 (3-5 anni)** – differenziazione subordinata (distinzione tra bassotto, pastore tedesco, barboncino).

Asimmetria nella generalizzazione: i bambini mostrano pattern asimmetrici di over-generalizzazione. **Over-extension verso l'alto** (rara): un bambino che conosce “cane” raramente chiama un gatto “animale” invece di cercare un termine più specifico. **Over-extension verso il basso** (comune): un bambino che conosce “cane” chiama spesso tutti i cani “cane”, senza distinguere le razze. Questo suggerisce che il livello base è il **punto di ancoraggio cognitivo** da cui si espandono sia generalizzazioni (verso l'alto) sia specificazioni (verso il basso).

5.4 Definizioni Moderne del Basic Level

Basic English di Ogden (1930): C.K. Ogden propose un sistema radicale – ridurre l'intero vocabolario inglese a **850 parole fondamentali**, basato su **minimalismo lessicale** (850 parole sufficienti per esprimere qualsiasi concetto), **composizionalità** (concetti complessi si costruiscono combinando parole base), **universalità** (le parole base sono simili tra lingue diverse). Esempio: *physician* (medico, termine avanzato) → in Basic English “person who makes sick people well”.

Critiche a Ogden: eccessivamente rigido e prescrittivo; non tiene conto della variazione contestuale; alcune parafrasi diventano innaturalmente lunghe; ignora la dimensione pragmatica (registro, formalità).

Definizioni di Basic Level nella letteratura recente:

Approccio	Definizione
Psicolinguistico	Parole apprese precocemente in L1, prioritarie in L2
Cognitivo	Termini che massimizzano distintività percettiva/min. sforzo
Frequenziale	Parole nel top 10% per frequenza in corpora bilanciati
Comunicativo	Lessico necessario per la “sopravvivenza sociale”
Computazionale	Parole con alta inter-annotator agreement, bassa ambiguità

Basic Vocabulary per la sopravvivenza sociale (prospettiva influente nei contesti L2): insieme minimo di parole necessarie per soddisfare bisogni primari (cibo, alloggio, salute), interagire in contesti quotidiani, comprendere istruzioni e segnaletica essenziali. Esempi per categoria: **Salute** (medico, ospedale, dolore, medicina, aiuto); **Cibo** (acqua, pane, mangiare, fame, ristorante); **Alloggio** (casa, letto, dormire, bagno, porta); **Tra-sporti** (autobus, treno, biglietto, fermare, andare); **Emergenze** (polizia, pericolo, perso, chiamare, numero). Questa prospettiva enfatizza la dimensione pragmatica e funzionale del basic level.

5.5 Proprietà e Fenomenologia del Basic Level

Il legame termine-concetto: non sempre 1:1. Polisemia e multi-funzionalità: parole di base possono riferirsi a concetti avanzati in domini specifici. Es. *cane*: senso base = “mammifero domestico della famiglia dei canidi”; senso avanzato (armeria) = “nelle armi da fuoco, barretta d'acciaio cilindrica che percuote il detonatore provocando l'accensione della carica di lancio”. **Implicazione:** il basic level è **domain-dependent** – *cane* è basic nel senso zoologico quotidiano ma advanced/tecnico nel dominio armiero.

Lacune lessicali (lexical gaps): alcune lingue mancano di termini di livello base per concetti comuni in altre culture. Esempio (colori): italiano ha *blu* (base); il russo distingue *siniy* (blu scuro) vs *goluboy* (blu chiaro) – due termini base distinti; alcune lingue amazzoniche hanno un solo termine per blu+verde. **Conseguenza:** il basic level **non è universale**, ma culturalmente e linguisticamente condizionato.

Soggettività del Basic Level: lo studio *Torrielli, Rapp, Di Caro (2023)*, “*How Shall a Machine Call a Thing?*” ha dimostrato sperimentalmente che la classificazione basic/advanced è **soggettiva**: annotatori umani mostrano significativa variabilità nel giudicare quali termini siano di base.

Fonti di variabilità: (1) **background educativo** (un botanico considera “quercia” basic, un non-esperto preferisce “albero”; un informatico usa “server” come basic, altri preferiscono “computer”); (2) **età** (bambini: vocabolario base più ristretto; adulti: categoria intermedia più ampia; anziani: possibile ritorno a vocabolario più semplice); (3) **contesto d’uso** (conversazione informale: preferenza per basic; scrittura formale: tendenza verso advanced; comunicazione specialistica: termini tecnici diventano “basic” nel dominio); (4) **L1 vs L2** (parlanti nativi: intuizione naturale del basic level; apprendenti L2: possono sovrastimare la complessità di parole comuni).

Implicazioni per l’annotazione: nei dataset annotati per basicness, l’inter-annotator agreement (IAA) è tipicamente **moderato** (Cohen’s $\kappa = 0.4-0.6$). Strategie di mitigazione: usare annotatori multipli (5-10 per item), definire linee guida chiare con esempi, calcolare consenso maggioritario, identificare “borderline cases” per analisi separate.

La definizione di “Advanced” per esclusione: a differenza del basic level, i termini advanced non hanno una definizione positiva consolidata: $\text{Advanced} = \neg \text{Basic}$ – tutto ciò che non è basic è, per definizione, advanced. Questa definizione negativa è problematica perché: (1) **assume dicotomia** (ignora la gradualità – alcune parole sono “moderatamente complesse”); (2) **manca granularità** (non distingue tra tecnico, formale, raro, arcaico); (3) **non cattura domini** (“mitocondrio” è advanced in generale ma basic in biologia). **Proposta alternativa:** modellare la basicness come **continuum** [0, 1] piuttosto che dicotomia – es. 1.0: cane, casa, mangiare, acqua; 0.7: veicolo, abitazione, consumare, liquido; 0.4: quadrupede, domicilio, ingerire, sostanza idrica; 0.1: canide, dimora, deglutire, H_2O .

5.6 Applicazioni del Basic Level in NLP

Text Simplification (l’applicazione più diretta): pipeline – identificare termini advanced (es. *dermatologo*, *dermatite atopica*, *allergeni ambientali*), sostituire con equivalenti basic (*dermatologo* → “medico della pelle”; *dermatite atopica* → “irritazione della pelle”; *allergeni ambientali* → “cose nell’aria che causano allergie”). Sfide: *loss of precision* (la semplificazione può perdere informazione), *verbosity* (le parafrasi basic sono spesso più lunghe), *naturalness* (sostituzioni automatiche possono risultare innaturali), *domain adaptation* (in contesti medici, “dermatologo” potrebbe essere necessario).

Second Language Learning: sequenziamento del vocabolario per livelli CEFR – A1 (500-800 parole, core basic), A2 (1000-1500, basic esteso), B1 (2000-3000, transizione basic-intermediate), B2 (4000-5000, semi-advanced), C1 (6000-8000, advanced+idiomatico), C2 (10000+, near-native). Sistemi NLP per L2 possono fare *vocabulary profiling* (assegnare livelli di difficoltà ai testi), *personalized exercises*, *reading assistance* (glossare automaticamente termini advanced), *writing feedback*.

Word Sense Disambiguation (WSD): ipotesi – i sensi basic di parole polisemiche sono più frequenti e più facili da disambiguare. Esempio *bank*: Senso 1 base (istituto finanziario, 80% occorrenze); Senso 2 advanced (riva del fiume, 15%); Senso 3 advanced (inclinazione aereo, 5%). Strategia: dare priorità a sensi basic in assenza di contesto disambiguante.

Machine Translation (MT): preservare il livello di registro – “The doctor examined the patient” (source basic) → Traduzione 1 (preserva basic): “Il medico ha visitato il paziente”; Traduzione 2 (eleva registro): “Il clinico ha esaminato il malato”. Un sistema MT sensibile al basic level dovrebbe preferire la Traduzione 1 in contesti generali. **Out-of-Vocabulary (OOV) handling:** se basic, alta priorità per trovare traduzione (probabile gap nel modello); se advanced, può usare translitterazione o parafrasi esplicativa.

Summarization: i riassunti dovrebbero privilegiare vocabolario basic per massimizzare accessibilità – un riassunto basic-oriented sostituisce termini advanced con equivalenti basic, privilegiando chiarezza su concisione.

Health Communication e Language Disorders: **afasia** (pazienti con disturbi linguistici post-ictus beneficiano di vocabolario semplificato); **dislessia** (parole basic facilitano la decodifica fonologica); **comunicazione medico-paziente** (evitare gergo medico migliora la comprensione); **informed consent** (documenti legali-medici devono essere accessibili).

5.7 Metriche e Risorse Computazionali

Come misurare la Basicness – sei approcci:

1. **Frequenza:** $\text{Basicness}_{\text{freq}}(w) = \frac{\text{count}(w)}{\text{total_tokens}}$, normalizzata con $\log(\text{count}(w) + 1)$ per ridurre skewness. Pro: facile da calcolare, buona correlazione con giudizi umani, risorsa-agnostica. Contro: dipendenza dal corpus (news vs conversazione), non distingue polisemia, parole funzionali (*di*, *a*, *il*) dominano.
2. **Age of Acquisition (AoA):** età media a cui una parola viene appresa. $\text{Basicness}_{\text{AoA}}(w) = \frac{1}{\text{AoA}(w)}$. Dataset di Kuperman et al. (2012): *mamma* AoA=1.2 anni (molto basic); *dinosauro* AoA=3.5 anni (basic); *fotosintesi* AoA=11 anni (advanced).
3. **Concreteness:** grado di tangibilità/percepibilità, scala [1,5] – 5 (massima): pietra, mela, cane; 3 (intermedia): amore, libertà, scienza; 1 (minima): ontologia, paradigma, epistemologia. Le parole concrete tendono a essere basic.
4. **Word Length:** $\text{Basicness}_{\text{len}}(w) = \frac{1}{\text{len}(w)}$. Le parole basic sono tipicamente 4-6 caratteri in italiano/inglese.
5. **Semantic Neighborhood Density:** numero di sinonimi/termini correlati. Parola basic (*cane*): molti sinonimi/iponimi (can, cagnolino, cucciolo, bastardo, randagio, bassotto...) → alta densità. Parola advanced (*coleottero*): pochi sinonimi generici (insetto, scarafaggio) → bassa densità.
6. **Composite Score:** $\text{Basicness}(w) = \alpha \cdot \log(\text{freq}) + \beta \cdot \frac{1}{\text{AoA}} + \gamma \cdot \text{concr} + \delta \cdot \frac{1}{\text{len}}$, dove $\alpha, \beta, \gamma, \delta$ sono pesi appresi da annotazioni umane.

Risorse e Dataset: WordNet (organizzazione gerarchica per livelli tassonomici); BabelNet (WordNet multilingue, studi cross-linguistici); AoA Norms (età di acquisizione per migliaia di parole); Concreteness Ratings (~40k parole inglesi); SUBTLEX (frequenze basate su sottotitoli, più rappresentative del parlato); CEFR Vocabulary Lists; Simple Wikipedia (versione semplificata con vocabolario controllato).

5.8 Esperimenti e Annotazione

Task di annotazione Basic/Advanced – Protocollo: (1) **selezione stimoli** (campionare parole da fasce di frequenza diverse, bilanciare parti del discorso, includere concrete e astratte); (2) **linee guida per annotatori** – “Una parola è BASIC se: la useresti parlando con un bambino di 6 anni; è una delle prime 1000 parole che impareresti in una lingua straniera; può essere spiegata senza usare termini tecnici. Altrimenti, è ADVANCED.”; (3) **formato:** scala binaria (BASIC/ADVANCED), scala Likert (1-5), o scala continua (slider 0-100); (4) **calcolo agreement**.

Cohen’s Kappa per 2 annotatori: $\kappa = \frac{P_o - P_e}{1 - P_e}$, dove P_o = proporzione di accordo osservato, P_e = proporzione di accordo atteso per caso. Interpretazione: $\kappa < 0.2$ poor; 0.2-0.4 fair; 0.4-0.6 moderate; 0.6-0.8 substantial; ≥ 0.8 almost perfect. **Risultati tipici:** $\kappa = 0.45$ -0.65 per basicness (moderate agreement).

Modelli supervised per Basicness Prediction: feature lessicali (lunghezza, affissi derivazionali, POS tag), corpus-based (log-frequenza, document frequency, distribuzione su registri), psicolinguistiche (AoA, concreteness, imageability), semantiche da WordNet (profondità, n. iperonimi/iponimi, n. sensi), embeddings (Word2Vec/FastText, BERT contestualizzato). Modelli: Logistic Regression (baseline interpretabile), Random Forest, Gradient Boosting/XGBoost (tipicamente le migliori performance), Neural Networks (MLP, BERT fine-tuned). Performance tipiche: Accuracy 75-85%, F1-score 0.72-0.82.

Possibile progetto di laboratorio: Ranking di Synset in WordNet. Approccio graph-based: WordNet è un grafo diretto (nodi=synset, archi=relazioni semantiche). Idea: synset basic sono quelli “centrali” nel grafo.

PageRank-based Basicness: $PR(s) = (1 - d) + d \sum_{s' \in \text{in}(s)} \frac{PR(s')}{|\text{out}(s')|}$, dove $\text{in}(s)$ = synset che puntano a s (iperonimi), $\text{out}(s')$ = synset a cui s' punta (iponimi), d = damping factor (0.85). Ipotesi: synset con alto PageRank sono più basic.

Approccio supervised: (1) annotare un subset di synset per basicness; (2) estrarre feature per ogni synset (depth in hierarchy, number of hyponyms, frequency of lemmas, gloss length/complexity); (3) addestrare un regressore synset $\rightarrow$ basicness score $\in [0, 1]$; (4) propagare predizioni a tutto WordNet.

5.9 Sfide Aperte e Direzioni Future

Multilinguallità: il basic level è universale o language-specific? **Ipotesi universalista:** concetti basic (cibo, famiglia, corpo) sono simili tra lingue. **Evidenza relativista:** lessicalizzazioni variano (colori, parentela, spazio). Serve ricerca comparativa su larga scala con annotazioni parallele.

Dinamicità temporale: il basic vocabulary evolve nel tempo – “computer” (advanced negli anni '60 $\rightarrow$ basic oggi), “smartphone” (neologismo 2007 $\rightarrow$ basic entro 2015), “telegrafo” (basic nel 1900 $\rightarrow$ arcaico oggi). Sfida: modelli che si adattano dinamicamente, tracciando l'evoluzione diacronica.

Context-Dependent Basicness: la stessa parola può essere basic o advanced a seconda del contesto – “virus”: contesto medico quotidiano = BASIC (“Ho preso un virus”); contesto virologico specialistico = può essere basic tra esperti ma advanced per il pubblico generale; contesto informatico = BASIC (“virus nel computer”). Direzione futura: modelli contestuali che predicono basicness dato (parola, contesto).

Granularità – oltre la dicotomia: basic/advanced è una semplificazione; serve modellazione più sfumata: multi-class (BASIC/INTERMEDIATE/ADVANCED/TECHNICAL), continuous (score $\in [0, 1]$), multi-dimensional (frequenza d'uso, complessità concettuale, registro stilistico, dominio di specializzazione).

Integrazione in LLM: i LLM potrebbero beneficiare di conoscenza esplicita sul basic level – *controllable generation* (generare testo con vocabolario di livello specificato), *adaptive explanation* (spiegare concetti complessi usando termini progressivamente più basic), *audience adaptation*. Approccio possibile: fine-tuning con reinforcement learning, premiando output che rispettano vincoli di basicness.

5.10 Conclusioni

Ricapitolazione: (1) **fondamenti teorici** – da Roger Brown (1958) a Eleanor Rosch, il basic level è un principio cognitivo fondamentale; (2) **proprietà** – brevità, concretez-

za, alta frequenza, facilità di apprendimento; (3) **collocazione gerarchica** – il basic level come “middle level” in tassonomie concettuali; (4) **variabilità** – soggettività, dipendenza dal contesto, non-universalità; (5) **applicazioni NLP** – text simplification, L2 learning, MT, WSD, summarization, health communication; (6) **metriche** – frequenza, AoA, concreteness, lunghezza, approcci compositi; (7) **modellazione computazionale** – classificazione supervised, ranking in WordNet, approcci graph-based.

Il paradosso del Basic Level: è simultaneamente **universale** (pattern simili emergono in tutte le lingue e culture) e **particolare** (le specifiche lessicalizzazioni sono language-specific); **oggettivo** (correlazioni robuste con metriche misurabili: frequenza, AoA) e **soggettivo** (variabilità significativa nei giudizi individuali); **stabile** (parole core restano basic attraverso generazioni) e **dinamico** (neologismi e arcaismi spostano continuamente i confini). Questa tensione riflette la natura stessa del linguaggio: un sistema contemporaneamente strutturato e fluido, universale e culturalmente situato, biologicamente radicato e socialmente costruito.

Implicazioni per NLP: non tutte le parole hanno lo stesso status cognitivo e comunicativo. Sistemi NLP che ignorano questa stratificazione lessicale rischiano di produrre output innaturali, inaccessibili o pragmaticamente inappropriati. Incorporare consapevolezza del basic level può migliorare accessibilità, naturalezza e adattività dei sistemi.

Domande di riflessione: Il basic level è una proprietà delle parole o dei concetti? Quanto è soggettiva la basicness – esiste un basic level “oggettivo” o solo un’approssimazione statistica? Come cambia il basic level con l’expertise (per un medico, “ECG” è basic – relativo alla comunità di pratica)? I LLM hanno una nozione implicita di basic level, o la simulano soltanto? È possibile una lingua con sole parole basic (il Basic English di Ogden, 850 parole, è davvero sufficiente)? Come evolverà il basic level con l’AI?

Bibliografia essenziale: Brown (1958) *How shall a thing be called?*; Rosch et al. (1976) *Basic objects in natural categories*; Lakoff (1987) *Women, Fire, and Dangerous Things*; Ogden (1930) *Basic English*; Nation (2001) *Learning Vocabulary in Another Language*; Coxhead (2000) *A new academic word list*; Kuperman et al. (2012) *Age-of-acquisition ratings for 30,000 English words*; Brysbaert et al. (2014) *Concreteness ratings for 40 thousand English word lemmas*; **Torrielli, Rapp, Di Caro (2023), “How Shall a Machine Call a Thing?”** (studio del professore sulla soggettività della classificazione basic/advanced).

6 Pre-LLM NLP: dal Text Mining in poi

6.1 Introduzione: Perché Studiare il Pre-LLM

Quando oggi interroghiamo ChatGPT, Claude o GPT-4, tendiamo a dare per scontato ciò che sembrano capacità quasi “magiche”. Ma ogni rivoluzione tecnologica ha radici profonde. Prima dell’avvento degli LLM – databile simbolicamente al 2017 con “Attention is All You Need” di Vaswani, ma che esplode realmente nel 2022-23 con ChatGPT – esisteva un intero universo di tecniche, filosofie e paradigmi che ha reso possibile quella rivoluzione: il **Pre-LLM NLP**.

Questo capitolo non è un museo di tecnologie obsolete. È una mappa genealogica che traccia come siamo arrivati fin qui – e perché molte di quelle “vecchie” tecniche non sono affatto scomparse, ma continuano a vivere, spesso nascoste sotto la superficie dei sistemi moderni che usiamo quotidianamente. Per capire davvero cosa significhi “fare NLP” oggi, dobbiamo chiederci: *cosa abbiamo guadagnato con gli LLM? E cosa, forse, abbiamo perso lungo la strada?*

Il Text Mining: quando i dati divennero protagonisti. Negli anni '80-'90, mentre la linguistica computazionale classica si concentrava su grammatiche formali scritte a mano da esperti, una comunità parallela iniziò a guardare al testo in modo radicalmente diverso: non come manifestazione di regole astratte da codificare, ma come **fonte di dati** da cui estrarre pattern, tendenze, conoscenza. Nacque così il Text Mining: applicazione delle tecniche del Data Mining (nate per database strutturati) al linguaggio naturale non strutturato. Era il trionfo del **bottom-up sul top-down**, dell'empirismo sul razionalismo, dell'induzione sulla deduzione – e funzionava, su milioni di documenti, su lingue diverse, su domini che nessun linguista avrebbe potuto codificare manualmente.

6.2 Una Storia in Quattro Atti: l'Evoluzione del NLP

Atto I – L'Era Simbolica (1950s-1980s): il sogno della macchina pensante. Se il linguaggio è governato da regole (Chomsky), allora possiamo codificare quelle regole e costruire macchine che “capiscono” il linguaggio, non lo processano superficialmente. È l'epoca di **ELIZA** (Weizenbaum, 1966), il primo chatbot, che simulava uno psicoterapeuta rogersiano con pattern matching e riformulazioni (“Mi sento triste” → “Perché ti senti triste?”) – nessuna comprensione reale, solo manipolazione sintattica, eppure molti utenti si convincevano di parlare con un essere pensante. È anche l'epoca di **SHRDLU** (Winograd, 1970), che manipolava blocchi virtuali rispondendo a comandi in inglese naturale – funzionava perfettamente in quel mondo artificiale ristretto, ma appena fuori da quel dominio il sistema crollava, non generalizzava. **Limite:** l'approccio simbolico era elegante e teoricamente solido, ma tremendamente fragile – ogni eccezione grammaticale richiedeva una nuova regola scritta a mano. Eredità viva oggi: l'idea di parsing sintattico, rappresentazione della conoscenza in strutture formali, inferenza logica su proposizioni estratte dal testo (es. i moderni knowledge graph).

Atto II – L’Era Statistica (1990s-2010s): quando i numeri presero il sopravvento. Gli anni '90 portano processori più potenti, memorie più capienti, e soprattutto: **dati**. Il web esplode da poche migliaia di pagine a miliardi. Nuovo paradigma: invece di codificare faticosamente le regole, lasciare che i modelli le **imparino** automaticamente dai dati. Nascono gli **Hidden Markov Models** per il PoS tagging (basta osservare abbastanza esempi, non servono più regole esplicite per decidere se “bank” è sostantivo o verbo); le **Conditional Random Fields** per la NER (il sistema impara da solo che sequenze come “Barack Obama” tendono a essere nomi di persona). Nascono i classificatori testuali: **Naive Bayes** (assume ingenuamente l’indipendenza tra parole – un’assunzione palesemente falsa – ma distingue spam da email legittime con accuratezza sorprendente); **Support Vector Machines** (trovano iperpiani ottimali in spazi a decine di migliaia di dimensioni). E nasce, forse più importante di tutto, l’idea che il significato delle parole possa essere catturato dalla loro **distribuzione**: TF-IDF dice che una parola è importante in un documento se appare spesso lì ma raramente altrove; le matrici di co-occorrenza dicono che due parole sono semanticamente simili se tendono a comparire negli stessi contesti, anche se non compaiono mai nella stessa frase. Questa epoca inventa anche il concetto stesso di **benchmark**: dataset standardizzati, metriche quantitative riproducibili (precision, recall, F1-score). **Limite**: tutto resta ancora **superficiale**. TF-IDF non capisce che “car” e “automobile” sono sinonimi. Le bag-of-words ignorano completamente l’ordine (“dog bites man” e “man bites dog” sono identiche).

Atto III – L’Era Neurale Pre-Transformer (2010-2017): il significato come geometria. Intorno al 2010 le reti neurali – esistenti da decenni ma abbandonate dopo l’“AI winter” degli anni '80 – tornano prepotentemente sulla scena, grazie a backpropagation efficiente su GPU sempre più potenti. Portano con sé un’idea rivoluzionaria: il significato delle parole può essere catturato come **vettori densi** in uno spazio geometrico ad alta dimensionalità – non più vettori sparsi one-hot di 100.000 dimensioni dove ogni parola è ortogonale a tutte le altre, ma **embeddings densi** di 300 dimensioni dove la similarità semantica diventa prossimità geometrica. **Word2Vec** (Mikolov, 2013) diventa un fenomeno culturale: l’idea è disarmante per semplicità – addestrare una rete a predire parole da contesto (o viceversa) – e nel processo la rete impara rappresentazioni che catturano relazioni semantiche sorprendenti: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$, una proprietà *emergente* genuina, non un trucco. Il significato non è più una lista di definizioni in un dizionario, né un nodo in un’ontologia costruita a mano: è una **posizione** in uno spazio geometrico, e le relazioni semantiche sono **direzioni** in quello spazio – un cambio di paradigma profondo, dalla semantica discreta alla semantica continua, dalla logica alla geometria. Arrivano le **LSTM**, reti ricorrenti che “ricordano” dipendenze lunghe; arrivano i meccanismi di **attention**. Ma siamo ancora in un mondo dove i task sono separati: un modello per traduzione, uno per sentiment analysis, uno per QA – non c’è ancora l’idea di un modello davvero universale.

Atto IV – L’Era Transformer e LLM (2017-presente): l’unificazione. Nel 2017 arriva “Attention is All You Need” (Vaswani et al.): non servono più reti ricorrenti complesse, basta attention – la capacità di metter in relazione ogni parola con ogni altra parola dell’input, in modo parallelo, efficiente, scalabile a sequenze lunghissime. I Transformer scalano in modo che le architetture precedenti non potevano: più parametri, più dati, più compute producono miglioramenti costanti, senza plateau (almeno non ancora trovato). Nel 2018 BERT introduce il **masked language modeling**: pre-training massiccio + fine-tuning leggero, e batte ogni benchmark esistente, spesso di tanto. Ma è **GPT-3** (2020), con 175 miliardi di parametri su 45TB di testo, a cambiare le regole del gioco: mostra **few-shot learning** – imparare nuovi task da pochi esempi nel prompt, senza fine-tuning, senza gradient updates, solo dalla capacità di riconoscere pattern e adattarli al contesto fornito. E quando arrivano ChatGPT (2022), GPT-4 (2023), Claude, vediamo qualcosa che nessuno aveva davvero anticipato: conversazione fluente, contestuale, che sembra “capire” intenzioni e sfumature.

Sembra. Quella parola è cruciale. Sotto la superficie di questa fluidità impressionante restano domande profonde senza risposta definitiva: questi modelli “capiscono” davvero il linguaggio? O sono pattern matcher incredibilmente sofisticati che hanno memorizzato così tanti esempi da poter approssimare comprensione senza possederla realmente? È una domanda filosofica che forse trascende la possibilità di risposta empirica.

Il quinto atto ancora da scrivere: oggi gli LLM dominano titoli di giornale, conferenze, investimenti, sembrano il futuro inevitabile del NLP. Ma sotto questo entusiasmo, una realtà più sfumata: i motori di ricerca di Google usano ancora BM25 (tecnica statistica anni '90) per il retrieval iniziale, prima di chiamare reranker neurali; molte aziende usano ancora SVM o Naive Bayes per classificazione produttiva, non per ignoranza ma perché servono decisioni interpretabili, auditabili, con latenza sotto il millisecondo; le pipeline di sentiment analysis su larga scala usano ancora TF-IDF perché processare 10 milioni di tweet con GPT-4 costerebbe decine di migliaia di dollari e giorni. **Forse la storia del NLP non è una successione di rivoluzioni che cancellano il passato, ma una stratificazione:** ogni epoca aggiunge un layer di capacità, senza necessariamente rimuovere quelli precedenti. Ogni tecnica trova la sua nicchia ecologica dove eccelle.

Dove si colloca il Text Mining: si posiziona decisamente nel campo bottom-up, nato come reazione pragmatica alla fragilità dei sistemi simbolici – “cosa possiamo fare con milioni di documenti e tecniche statistiche, senza costruire ontologie a mano?”. Sorprendentemente tanto: raggruppare per tema (clustering), classificare apprendendo pattern dai dati, estrarre termini chiave, riassumere documenti – tutto attraverso metodi statistici su frequenze, co-occorrenze, distribuzioni. Ma **il Text Mining intelligente non ignora completamente la conoscenza linguistica:** usa lemmatization, POS tagging, dependency parsing dove serve aggiungere valore reale misurabile. Non è purismo metodologico, è ingegneria responsabile – capire *quando* ciascun approccio è appropriato, *dove* i suoi punti di forza risolvono problemi reali, e *come* combinarli intelligentemente invece di vederli come filosofie incompatibili in guerra permanente.

6.3 Dalle Frequenze al Significato: l’Intuizione Distribuzionale

“You Shall Know a Word by the Company It Keeps” (Firth, 1957). Questa frase cattura un’idea profonda ma semplice: il significato di una parola non sta in qualche definizione astratta nel cielo platonico delle idee, ma nelle sue **relazioni d’uso** con altre parole nel linguaggio reale. Se due parole compaiono sistematicamente negli stessi contesti – se possono essere sostituite l’una con l’altra in molte frasi mantenendo coerenza semantica – allora probabilmente hanno significati simili. “Car”/“automobile”, “dog”/“canine”, “happy”/“joyful” non compaiono MAI nella stessa frase (sarebbero ridondanti), ma compaiono in frasi molto simili, con verbi simili, aggettivi simili, contesti simili. **Paradossalmente**, per capire la similarità semantica non si deve guardare alla co-occorrenza diretta (rarissima tra sinonimi), ma alla similarità dei *pattern* di co-occorrenza con *altre* parole.

Dal Vector Space Model agli embeddings: un viaggio concettuale. Gerard Salton (1974) propone di rappresentare ogni documento come un vettore in uno spazio ad alta dimensionalità, dove ogni dimensione corrisponde a un termine del vocabolario – la similarità diventa un problema geometrico (coseno dell’angolo tra vettori). TF-IDF raffina pesando i termini per distintività. Ma tutto questo opera nel paradigma **sparse**: vettori con 100.000 dimensioni dove il 99% sono zeri, e ogni dimensione ha un’interpretazione esplicita (corrisponde a una parola specifica). Il salto concettuale – che arriva negli anni 2000 con LSA e poi esplode con Word2Vec nel 2013 – è passare a rappresentazioni **dense**: vettori di sole 300 dimensioni, tutte non-zero, dove le dimensioni non corrispondono più a parole specifiche, ma sono “direzioni latenti” che il modello scopre automaticamente. E qui la magia: in questo spazio denso, “car” e “automobile” finiscono vicini anche se non co-occorrono mai direttamente, perché condividono pattern di contesto.

Cosa Abbiamo Guadagnato, Cosa Abbiamo Perso (1). Gli embeddings densi hanno portato: **cattura di sinonimi e relazioni semantiche** senza co-occorrenza diretta; **analogie emergenti** (king:queen = man:woman come parallelismo geometrico); **generalizzazione a parole mai viste**; **compressione dimensionale** (da 100.000 sparse a 300 dense); **transfer learning** (embeddings pre-addestrati riusabili con pochi dati). Ma abbiamo perso: **interpretabilità** (le matrici di co-occorrenza erano trasparenti, gli embeddings densi sono opachi – non puoi guardare la dimensione 47 e dire “ah, questa cattura l’animatezza”); **determinismo** (TF-IDF è completamente deterministico, Word2Vec ha randomness nell’inizializzazione e nel training); **verificabilità** (con TF-IDF potevi ispezionare la matrice e capire esattamente perché due documenti risultavano simili; con gli embeddings è un mistero dentro trasformazioni non-lineari); **controllo** (potevi modificare manualmente pesi TF-IDF, con embeddings neurali interventi manuali sono quasi impossibili). Questo pattern – guadagnare potenza ma perdere trasparenza – si ripeterà ancora più drammaticamente con BERT, GPT e gli LLM moderni: fino a che punto siamo disposti ad accettare opacità in cambio di capacità? Dipende dall’applicazione (raccomandare film su Netflix è diverso da concedere un prestito bancario o diagnosticare una malattia).

6.4 I Task del Text Mining: Cosa Cercavamo di Risolvere

6.4.1 Clustering: Scoprire Strutture Nascoste

Problema umano: hai 10.000 documenti, non sai quali temi contengano, non hai tempo di leggerli tutti. Il **clustering non supervisionato** scopre le categorie guardando solo le

similarità tra documenti, senza etichette. Negli anni '80, Stuart Lloyd (Bell Labs) propone **K-means**: sorprendentemente semplice – scegli k centroidi casuali, assegna ogni documento al centroide più vicino, ricalcola i centroidi, ripeti fino a convergenza. **Problema filosofico**: come sapere se un clustering è “buono”? Nella classificazione supervisionata hai la verità oggettiva (le etichette); nel clustering, manca – puoi misurare compattezza e separazione, ma sono proprietà geometriche che catturano solo *approssimativamente* la “bontà semantica”. Due sistemi di clustering possono produrre partizioni diverse, entrambe geometricamente valide, ma semanticamente una ha senso per gli umani e l'altra no – non esiste un modo algoritmico puramente automatico per decidere quale sia “migliore” senza giudizio umano.

6.4.2 Classificazione: Apprendere da Esempi Etichettati

Qui la situazione è più chiara: hai esempi etichettati (5.000 email marcate spam/ham), vuoi un sistema che impari i pattern e classifichi email nuove. Negli anni '90, McCallum dimostra che **Naive Bayes** (assunzione ingenuamente falsa di indipendenza tra parole) funziona sorprendentemente bene – cattura comunque segnali statistici forti (“viagra”, “lottery” compaiono molto più in spam). Poi arrivano le **Support Vector Machines** (Joachims, 1998), che diventano lo standard per un decennio. **Cosa hanno in comune questi metodi classici con i classificatori neurali moderni?** Più di quanto sembri: il principio fondamentale resta identico – **apprendere pattern da dati annotati**. Ciò che è cambiato è la capacità di catturare pattern non-lineari complessi (BERT “capisce” che “not bad” è positivo anche se contiene “bad”, perché ha appreso la negazione come trasformazione semantica; Naive Bayes vedrebbe solo “bad” e classificherebbe probabilmente negativo). Ma il trade-off interpretabilità vs. performance persiste: con Naive Bayes/SVM lineari puoi ispezionare i pesi; con BERT fine-tuned è una scatola nera da 110 milioni di parametri. Per applicazioni regolamentate (medicina, finanza, legale) questa opacità è spesso inaccettabile, e SVM/Logistic Regression restano in uso per necessità di accountability, non per ignoranza.

6.4.3 Summarization: l'Essenza del Testo

I primi sistemi **estrattivi** (anni '90-2000) selezionavano le frasi più “salienti” (euristiche: prima frase di ogni paragrafo, frasi con TF-IDF alto, frasi lunghe) e le concatenavano. Nel 2004, Rada Mihalcea propone **TextRank**: tratta le frasi come nodi di un grafo con archi pesati dalla similarità lessicale, applica PageRank per trovare frasi “centrali”. Limite fondamentale: i metodi estrattivi potevano solo **selezionare e riordinare** – non potevano parafrasare, condensare, fondere informazioni da frasi multiple in una frase nuova più concisa. I metodi **astrattivi** (reti neurali seq2seq, 2015-2017) rimasero fantascienza fino a BART e T5 (2019-2020), pre-addestrati su enormi quantità di testo. Ma abbiamo perso qualcosa: i metodi estrattivi erano **fedeli per costruzione** (copiavano frasi esatte dal testo); i metodi astrattivi possono generare riassunti più fluidi ma anche dire cose che il documento originale non conteneva. È una feature o un bug? Dipende: per legal document summarization è un bug; per news aggregation forse è una feature.

6.4.4 Information Retrieval: dal Boolean Search al Semantic Search

Storia affascinante che riflette tutta l'evoluzione del NLP. Anni '60-'70: **Boolean search** (query come “(machine AND learning) OR (neural AND network)”, match rigido binario,

nessuna nozione di rilevanza parziale). Anni '70-'90: Salton introduce il VSM e il ranking graduato basato su cosine similarity; **BM25** di Robertson (anni '90) perfeziona pesando per distintività e normalizzando per lunghezza documento, diventando lo standard de facto per 25+ anni – e ancora oggi (2024) Google Search usa BM25 (o varianti) per il primo stage di retrieval su miliardi di pagine, perché è velocissimo, deterministico, comprensibile. Anni 2010, con Word2Vec/GloVe: emerge l'idea di **semantic search** – query “automobile” dovrebbe matchare documenti con “car” anche senza sovrapposizione di parole. **Dense retrieval** (2019-2020): ogni query/documento diventa embedding denso via BERT, similarity = dot product; cattura sinonimia/paraphrasing che BM25 perdeva, ma calcolare similarità contro tutti i documenti non scala oltre 10M documenti senza approssimazioni (ANN). **Risultato**: i sistemi moderni usano **pipeline ibride** – (1) BM25 recupera 1000 candidati (CPU, millisecondi); (2) BERT cross-encoder reranka top-100 (GPU, secondi); (3) GPT-4 genera snippet o risponde direttamente (opzionale). Tecniche di epoche diverse coesistono in simbiosi, ciascuna eccellendo dove le altre sono debili – non sostituzione, ma stratificazione.

6.4.5 Topic Modeling: Scoprire Temi Nascosti

Il ponte più evidente tra approccio classico e moderno. Diverso dal clustering: non vuoi solo raggruppare documenti simili, vuoi **caratterizzare** quei gruppi con liste di parole chiave interpretabili.

L'era classica – LDA: Latent Dirichlet Allocation (Blei, Ng, Jordan, 2003) – idea probabilistica elegante: ogni documento è una “miscela” di topic, ogni topic è una distribuzione di probabilità su parole (es. un paper potrebbe essere 40% “machine learning”, 30% “computer vision”, 30% “NLP”). Limiti evidenti: opera su bag-of-words (ignora ordine/sintassi); non cattura sinonimia (“car”/“automobile” parole separate); richiede di specificare k a priori; interpretare i topic richiede ancora giudizio umano.

L'era moderna – BERTopic e Neural Topic Models: con gli embeddings neurali il paradigma cambia radicalmente – invece di modellare distribuzioni di parole su topic astratti, si parte da rappresentazioni semantiche dense dei documenti. **BERTopic** (Groo-tendorst, 2022) è paradigmatico, pipeline ibrida: (1) encoding neurale (BERT/Sentence-BERT); (2) UMAP riduce a 5-10 dimensioni preservando struttura locale; (3) HDBSCAN trova cluster di densità variabile, gestendo outliers; (4) class-based TF-IDF estrae parole più distintive per ogni cluster (non le più frequenti in assoluto, ma quelle che *distinguono* quel cluster dagli altri). Risultato: topic molto più coerenti semanticamente – documenti su “autonomous vehicles” e “self-driving cars” finiscono nello stesso cluster anche senza condividere parole.

Caso d'uso: 45.000 abstract NLP da ArXiv → BERTopic. Senza specificare nulla a priori, emergono topic come: *Topic 0*: translation, machine, neural, attention, encoder → Neural Machine Translation; *Topic 1*: sentiment, opinion, polarity, aspect → Sentiment Analysis; *Topic 2*: question, answer, retrieval, passage, squad → Question Answering; *Topic 3*: graph, knowledge, entity, relation, embedding → Knowledge Graphs. Emersi automaticamente dalla struttura semantica latente, interpretabili guardando le parole chiave.

Cosa Abbiamo Guadagnato, Cosa Abbiamo Perso (2): LDA vs. BERTopic. Guadagnato: coerenza semantica (topic più interpretabili di LDA); robustezza a sinonimi; gestione outliers (documenti fuori tema non forzati in cluster arbitrari); visualizzabilità (mappa concettuale via UMAP 2D); flessibilità (cambiare embedding model cambia granularità catturata). Perso: **interpretabilità probabilistica** (LDA dava distribuzioni esplicite “questo documento è 40% topic A”; BERTopic assegna cluster hard, senza probabilità); **modello generativo** (non puoi “campionare” nuovi documenti da un topic come con LDA); **trasparenza** (perché BERT ha messo questo documento in quel cluster? mistero); **costi computazionali** (encoding 100k documenti con BERT richiede GPU e ore; LDA scala meglio su CPU). Per collezioni massive dove velocità conta più di qualità (monitoring social media real-time), LDA può essere preferibile; per analisi profonde dove coerenza semantica è cruciale, i metodi neurali eccellono.

6.5 Conclusione: Cosa Abbiamo Imparato, Dove Stiamo Andando

Il paradosso del progresso: mentre celebriamo (giustamente) la rivoluzione degli LLM, rischiamo di dimenticare le fondamenta su cui quella rivoluzione è stata costruita – e di perdere qualcosa di prezioso. TF-IDF, BM25, Naive Bayes, K-means, TextRank non sono solo acronimi in paper polverosi: sono intuizioni profonde su come estrarre significato da simboli, su come misurare rilevanza senza “capire”, su come trovare struttura in dati non annotati. Quando BERT usa masked language modeling, sta facendo distributional semantics – la stessa idea di Firth, solo su scala massiccia. Quando GPT-4 fa in-context learning da pochi esempi, sta facendo una forma di k-NN nei suoi spazi latenti – come K-means ma infinitamente più sofisticato. **Il paradosso:** mentre i metodi specifici evolvono radicalmente, i **principi sottostanti** mostrano notevole continuità – l’ipotesi distribuzionale resta valida, il trade-off interpretabilità vs. potenza resta irrisolto, la necessità di valutazione empirica rigorosa resta centrale.

Cosa gli LLM hanno risolto (e cosa no). **Risolto (o quasi):** generalizzazione cross-task (un modello, mille applicazioni senza retraining); comprensione contestuale profonda (sfumature, sarcasmo, implicazioni); few-shot learning; generazione fluente; accessibilità (chiunque può “programmare” con linguaggio naturale). **Non risolto (o peggiorato):** **interpretabilità** (cosa ha imparato GPT-4 nei suoi 1.7 trilioni di parametri? mistero quasi completo); **controllabilità** (come garantire che non generi contenuti dannosi, falsi, biased? problema aperto); **efficienza** (TF-IDF classifica 100k documenti in secondi su laptop; GPT-4 richiede cluster GPU e milioni di dollari); **riproducibilità** (temperature sampling rende output stocastico); **verità e hallucination** (inventano fatti, citazioni con sicurezza disarmante); **costi economici ed ecologici** (training GPT-3 ha emesso CO₂ come cinque auto in tutta la loro vita); **concentrazione di potere** (solo poche corporation possono addestrare frontier models). Progresso innegabile, ma non panacea universale – e problemi classici (evaluation, bias, fairness, robustness) persistono o si amplificano.

La stratificazione, non la sostituzione: il progresso tecnologico nel NLP raramente è sostituzione completa; più spesso è **stratificazione**. Sistemi reali in produzione nel 2024: Google Search (Stage 1 BM25, Stage 2 reranker neurali, Stage 3 LLM per snippet); classificazione spam (Naive Bayes/SVM per latenza <10ms su milioni di email); sentiment analysis su larga scala (TF-IDF+Logistic Regression: poche ore, \$100 di compute, vs. \$50.000+ e giorni con GPT-4 per accuratezza extra che spesso non vale il costo 500×);

sistemi di raccomandazione (Word2Vec su sequenze di acquisti funziona ancora benissimo: “if it ain’t broke, don’t fix it”). Non è nostalgia, è **pragmatismo ingegneristico**: tool diversi sono ottimali per problemi diversi, constraint diversi, requirement diversi.

Quattro lezioni dal passato per il futuro:

1. **L’importanza delle assunzioni esplicite**: i metodi pre-LLM avevano un vantaggio sottovalutato – le loro assunzioni erano *esplicite* (TF-IDF assume che termini rari siano discriminativi; Naive Bayes assume indipendenza condizionale). Potevano essere sbagliate, ma sapevamo quali fossero – c’era trasparenza epistemologica. Con gli LLM, le assunzioni sono *implicite*, sepolte in miliardi di parametri ottimizzati da gradient descent stocastico: non sappiamo cosa ha davvero imparato il modello, quali bias ha assorbito, quali scorciatoie spurie usa invece di ragionamento profondo.
2. **Il valore della semplicità**: TF-IDF fu proposto nel 1972; cinquant’anni dopo è ancora usato in produzione perché è semplice, veloce, deterministico, comprensibile. Non è il metodo più accurato, ma queste proprietà valgono spesso più dell’accuratezza marginale che un modello complesso potrebbe dare – la complessità ha costi computazionali, manutentivi, di debugging, di deployment, ecologici.
3. **L’illusione della generalità**: ogni epoca del NLP ha avuto il suo “modello universale” che prometteva di risolvere tutto (anni ’80: sistemi esperti; anni ’90: SVM con kernel universali; 2010s: BERT pre-addestrato; 2020s: GPT-4 e few-shot prompting). Ogni volta entusiasmo genuino, ogni volta col tempo si scoprivano limiti e failure mode. GPT-4 è straordinariamente versatile, ma metodi specializzati battono ancora i LLM generici su task specifici (NER domain-specific con CRF; parsing sintattico formalmente corretto con parser neurali specializzati; classificazione binaria ad altissimo throughput con SVM ottimizzate).
4. **Il costo nascosto dei dati**: ogni generazione richiede più dati, più compute, più energia, fino a un punto di insostenibilità economica (solo poche corporation possono permetterselo) ed ecologica (emissioni paragonabili a paesi piccoli), e socialmente problematica (dati raschiati dal web senza consenso degli autori – questioni di copyright, privacy, equità). Per il futuro: investire in metodi **data-efficient** e **compute-efficient** (distillation, pruning, quantization), non solo modelli sempre più grandi.

Guardando avanti: il dialogo tra generazioni. Il modo migliore per pensare alla relazione tra Pre-LLM NLP e LLM non è come successione lineare (vecchio vs. nuovo, obsoleto vs. moderno), ma come **dialogo continuo** tra approcci complementari. Gli LLM eccellono in generazione fluente, adattamento rapido a nuovi task, comprensione contestuale profonda, ma sono opachi, costosi, stocastici, difficili da controllare. I metodi classici eccellono in efficienza computazionale, interpretabilità, controllabilità, determinismo, deployment leggero, ma sono rigidi e catturano solo pattern superficiali. Il futuro probabilmente sarà **ibridazione intelligente**: LLM per generare candidati creativi, metodi classici per validation e ranking; embeddings neurali per rappresentazione ricca, metriche classiche interpretabili per valutazione; few-shot prompting per prototipazione rapida, fine-tuning supervisionato per deployment produttivo robusto; neural retrieval per recall semantico, BM25 come fallback quando il neurale fallisce o per vincoli di latenza; GPT-4 per task complessi con alta tolleranza a errori, SVM per task semplici con zero tolleranza.

6.6 Laboratorio LAB-4: Topic Modeling con BERTopic

Obiettivo: costruire una pipeline di clustering e topic modeling su un dataset di articoli scientifici, usando embeddings neurali, riduzione dimensionale e algoritmi di clustering denso. L'obiettivo non è solo ottenere topic coerenti, ma capire *perché* certe scelte funzionano meglio di altre per questo specifico dominio. **Dataset:** ~45.000 abstract di paper NLP da ArXiv (huggingface.co/datasets/MaartenGr/arxiv_nlp), variabili **Abstracts** e **Titles**. Scelto perché: abbastanza grande da mostrare pattern statistici robusti, domain-specific (topic interpretabili), moderatamente challenging (linguaggio tecnico e sovrapposizioni tematiche).

Pipeline in 5 fasi (vedi anche Sez. 15.1 per l'implementazione concreta svolta e i risultati comparativi):

1. **Preprocessing** (opzionale ma consigliato): lowercase, rimozione caratteri speciali/numeri/URL, rimozione stopword domain-specific ("paper", "method", "propose"), lemmatization vs stemming vs nessuno. Domanda da esplorare: il preprocessing migliora davvero la coerenza, o gli embeddings neurali sono già robusti al rumore?
2. **Embedding dei documenti:** modello consigliato iniziale `thenlper/gte-small` (General Text Embeddings, ottimizzato per similarità semantica generale); alternative: `all-MiniLM-L6-v2` (più veloce, 384-dim), `all-mpnet-base-v2` (più lento, qualità superiore, 768-dim), modelli domain-specific (SciBERT-based).
3. **Riduzione dimensionale con UMAP:** parametri da sperimentare – `n_components` (5 vs 10 vs 15: più componenti = più informazione preservata ma clustering più difficile), `min_dist` (0.0 cluster compatti vs 0.1 più spazio), `metric` (cosine vs euclidean), `n_neighbors` (15 default vs 30 vs 50, controlla balance locale/globale).
4. **Clustering con HDBSCAN:** parametri critici – `min_cluster_size` (50 vs 100 vs 200: più alto = cluster più grandi/stabili ma perdita di granularità), `cluster_selection_method` ("eom"/Excess of Mass vs "leaf"), `min_samples` (controllo rumore).
5. **Topic Modeling con BERTopic:** integra embedding+UMAP+HDBSCAN, estrae keyword interpretabili con c-TF-IDF; esplorazione con `get_topic_info()`, `get_topic(n)`, `get_representative_docs(n)`.

Visualizzazione e validazione: `visualize_documents()` (proietta embeddings in 2D colorati per cluster – cercare cluster compatti e separati, outlier distribuiti o concentrati); `visualize_barchart()` (top parole per topic – validazione qualitativa: le parole hanno senso insieme?); `visualize_hierarchy()` e `visualize_heatmap()` (relazioni tra topic, utile per capire over-segmentation o under-segmentation).

Requisiti e deliverable: implementare la pipeline completa; sperimentare almeno 3 configurazioni diverse (embedding model, parametri UMAP, parametri HDBSCAN); per ogni configurazione documentare numero di topic, % outlier, top-5 parole per i 10 topic più popolati, tempo computazionale; validazione qualitativa su almeno 5 topic (esaminare documenti rappresentativi); confronto critico finale (quale configurazione produce topic più interpretabili, perché, quali trade-off). Bonus opzionali: dataset alternativo, query-based search, tracking temporale, confronto con LDA classico.

Suggerimenti pratici: iniziare piccolo (5k documenti per debug, poi scalare a 45k); visualizzare sempre (i numeri mentono, gli scatter plot rivelano se i cluster sono arbitrari); leggere documenti random da un cluster invece di fidarsi ciecamente delle parole chiave; pianificare i costi computazionali (embedding di 45k documenti può richiedere 10-30 minuti su GPU, ore su CPU); usare `random_state=42` per riproducibilità (UMAP e HDBSCAN hanno componenti stocastiche); variare un parametro alla volta per capire l'impatto individuale.

Domande di riflessione dalle dispense: L'approccio top-down (simbolico) e bottom-up (statistico) riflettono filosofie diverse sul linguaggio – quale è più “corretto”, o è la domanda stessa mal posta? TF-IDF cattura importanza senza “capire” significato; GPT-4 genera testo che sembra “capire” – cos'è davvero “capire”, è una proprietà binaria o un continuum? Cosa rende certi che gli LLM non siano solo l'ultimo capitolo di un'illusione ricorrente (di aver finalmente risolto il problema)? I metodi pre-LLM sono ancora usati in produzione non per ignoranza, ma per scelta – cosa dice questo sui limiti reali (non solo teorici) degli LLM? Se addestrare GPT-5 richiedesse emissioni di CO₂ equivalenti a una nazione di medie dimensioni, sarebbe giustificabile? Word2Vec ha mostrato che analogie semantiche emergono automaticamente da semplice predizione contestuale – cosa implica sulla natura del significato linguistico?

Bibliografia essenziale: Firth (1957) *A synopsis of linguistic theory*; Harris (1954) *Distributional structure*; Weizenbaum (1966) *ELIZA*; Winograd (1972) *Understanding natural language*; Salton et al. (1975) *A vector space model for automatic indexing*; Robertson & Walker (1994) *2-Poisson model – BM25*; Manning et al. (2008) *Introduction to Information Retrieval*; Mikolov et al. (2013) *Distributed representations of words and phrases*.

7 Retrieval-Augmented Generation: tra Memoria Esterna, Retrieval e LLM

7.1 Introduzione

I LLM sono in grado di generare testo fluido, coerente e spesso sorprendentemente informato. Tuttavia questa capacità nasconde una fragilità strutturale: il loro sapere è **implicito**, distribuito nei parametri, e soprattutto **non verificabile**. Un modello come GPT non possiede un archivio consultabile – non può “andare a controllare” una fonte, può solo produrre la continuazione più probabile di una sequenza in base a ciò che ha appreso durante il training. Quando il modello non ha conoscenza sufficiente su un argomento, **non si ferma**: continua comunque a generare. È qui che emergono le **allucinazioni**: risposte plausibili ma false, o risposte corrette ma obsolete – particolarmente critico nei domini specialistici (medico, legale) dove la conoscenza evolve rapidamente. La RAG nasce precisamente come risposta a questo limite: non tenta di rendere il modello “onnisciente”, ma introduce un principio diverso – **separare la capacità di generare linguaggio dalla capacità di accedere alla conoscenza**.

7.2 Il Problema dell'Aggiornamento

Esempio concreto: chiediamo a un LLM addestrato fino al 2023 informazioni su eventi del 2024. Il modello non può sapere cosa è accaduto dopo il training cutoff: può solo inventare una risposta plausibile (allucinazione), ammettere di non sapere (se opportunamente istruito), o generalizzare da pattern simili visti durante il training. **Nessuna di queste opzioni è soddisfacente** per applicazioni che richiedono precisione fattuale. La RAG risolve questo problema fondamentale: invece di affidarsi solo alla memoria parametrica del modello, introduce una memoria esterna aggiornabile, interrogabile e verificabile.

7.3 Dal Sapere Implicito al Sapere Accessibile

Un LLM tradizionale funziona come una **memoria compressa**: tutto ciò che “sa” è codificato nei pesi del modello, senza accesso selettivo (non possiamo interrogare direttamente i suoi parametri per estrarre un fatto specifico). La RAG introduce una seconda forma di memoria, esplicita e interrogabile. Due livelli: **memoria parametrica (interna)** – conoscenza distribuita nei parametri, appresa durante il pre-training, implicita, compressa, difficile da modificare; **memoria non-parametrica (esterna)** – documenti, database, collezioni testuali, esplicita, strutturata, facilmente aggiornabile.

Il punto cruciale è che la risposta non viene più generata esclusivamente dalla memoria interna, ma è il risultato dell'interazione tra queste due fonti. Questo porta a un cambiamento epistemologico profondo: *un modello non deve più sapere tutto. Deve sapere come trovare ciò che non sa.*

7.4 Information Retrieval: il Problema del Trovare

L'IR si occupa di un problema apparentemente semplice: dato un bisogno informativo, trovare i documenti più rilevanti in una collezione. Un sistema di IR non deve generare una risposta – deve *selezionare*, da una collezione di documenti, quelli più pertinenti. Questo introduce una tensione fondamentale tra due dimensioni: **Precision** (quanto i documenti recuperati sono rilevanti rispetto alla query) e **Recall** (quanta della conoscenza rilevante disponibile viene effettivamente recuperata).

$$\text{Precision} = \frac{\text{documenti rilevanti recuperati}}{\text{documenti totali recuperati}}, \quad \text{Recall} = \frac{\text{documenti rilevanti recuperati}}{\text{documenti rilevanti totali}}$$

Esiste un trade-off fondamentale: aumentare il recall spesso significa diminuire la precision, e viceversa.

7.4.1 Approcci Classici: TF-IDF e BM25

TF-IDF: assegna un peso a ogni termine in base a quanto frequentemente appare nel documento (TF) e quanto raramente appare nell'intera collezione (IDF) – parole comuni (“il”, “di”) hanno poco valore discriminante, parole rare e specifiche sono più informative. $\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log(N/\text{DF}(t))$.

BM25 è un'evoluzione di TF-IDF che introduce saturazione: dopo un certo punto, aumentare la frequenza di un termine non aumenta significativamente il suo peso. **Limite di entrambi:** non catturano il significato – una query su “auto” potrebbe non recuperare documenti che parlano di “automobile” o “veicolo”, anche se semanticamente equivalenti (**vocabulary mismatch**).

7.5 Semantica Distribuzionale e Retrieval Semantico

La semantica distribuzionale rappresenta le parole come vettori in uno spazio continuo, dove la vicinanza riflette similarità semantica. La stessa idea si estende ai documenti tramite **embeddings densi**. In un sistema di **dense retrieval**, sia la query che i documenti vengono proiettati nello stesso spazio vettoriale ad alta dimensionalità (768 o 1024 dimensioni): (1) la query diventa $\vec{q} \in \mathbb{R}^d$; (2) ogni documento diventa $\vec{d}_i \in \mathbb{R}^d$; (3) si calcola la similarità (tipicamente coseno: $\text{sim}(\vec{q}, \vec{d}) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|}$, o alternativamente distanza euclidea/dot product); (4) si selezionano i k documenti più vicini (top- k retrieval).

7.5.1 Modelli per Dense Retrieval

Sentence-BERT (SBERT): variante di BERT con architettura siamese/a triplette, per embeddings di frase confrontabili tramite coseno – risolve l'inefficienza computazionale di BERT per semantic search/clustering, riducendo il tempo da 65 ore a 5 secondi per 10.000 frasi su STS benchmark. **DPR (Dense Passage Retrieval)**: encoder BERT duali che mappano query e passaggi in spazio a 768 dimensioni; supera i metodi sparsi come BM25 del 9-19% in accuracy top-20 per open-domain QA. **ColBERT**: embeddings contestualizzati a livello di token (non documento); architettura “late interaction” con MaxSim, retrieval efficiente su grandi collezioni. **Contriever**: retrieval denso self-supervised con contrastive learning, senza dati supervisionati; competitivo con BM25 su benchmark BEIR. Questi modelli sono addestrati con un obiettivo **contrastivo**: massimizzare similarità tra query e documenti rilevanti, minimizzarla con documenti non rilevanti.

7.6 Architettura RAG: uno Schema Concettuale

La RAG è una composizione di due sistemi: un sistema di retrieval (IR) e un sistema generativo (LLM). Schema generale: Utente $\rightarrow$ Query $\rightarrow$ Retriever $\rightarrow$ Documenti $\rightarrow$ LLM $\rightarrow$ Risposta. Questo schema lineare nasconde una struttura più ricca, con **cinque componenti fondamentali**: (1) **Document Store** (la collezione di documenti); (2) **Indexing** (preprocessing e creazione di embeddings); (3) **Query Processing** (trasformazione della query in embedding); (4) **Retrieval** (selezione dei documenti più rilevanti); (5) **Generation** (sintesi della risposta dal contesto recuperato).

Esempio completo – “Quali sono i sintomi del diabete di tipo 2?”. **1. Encoding della query:** $\vec{q} = \text{Encoder}(\text{“Quali sono i sintomi...”})$. **2. Retrieval:** il sistema recupera i 3 documenti più simili (es. Doc 1 sim=0.89: “Il diabete di tipo 2 è caratterizzato da iperglicemia...”; Doc 2 sim=0.85; Doc 3 sim=0.82). **3. Costruzione del prompt:** “Contesto fornito: [Doc 1][Doc 2][Doc 3]. Domanda: [query]. Rispondi basandoti esclusivamente sul contesto fornito sopra.” **4. Generazione:** il modello produce una risposta sintetizzata dai 3 documenti – **non** è più una semplice continuazione linguistica, ma una sintesi guidata da evidenze documentali.

Varianti architetturali: **RAG-Sequence** (ogni documento recuperato genera una risposta separata, poi si aggregano); **RAG-Token** (a ogni step di generazione si può re-interrogare il retriever); **Self-RAG** (il modello decide autonomamente quando invocare il retrieval); **Iterative RAG** (si alternano cicli di retrieval e generazione per query complesse).

7.7 Chunking: il Problema della Granularità della Conoscenza

Non si lavora direttamente con documenti interi, ma con porzioni più piccole chiamate **chunk**, per tre motivi: gli LLM hanno limiti di contesto (context window); troppa informazione irrilevante degrada la qualità della risposta (effetto “dilution”); chunk più piccoli consentono retrieval più preciso. Ma questo introduce una nuova versione del problema della granularità, già visto nella semantica lessicale: *qual è la dimensione “giusta” di un’unità di significato?* Se i chunk sono troppo piccoli (singole frasi), si perde contesto (una frase isolata può essere ambigua); se troppo grandi (pagine intere), si introduce rumore e si rischia di superare il context window.

Strategie di chunking: (1) **chunking fisso** – dividere ogni N token/caratteri ($N \in [256, 512, 1024]$ tipicamente, con overlap 10-20%); semplice e veloce ma ignora la struttura del testo, può spezzare frasi/concetti; (2) **chunking strutturale** – usa i marcatori naturali del testo (paragrafi, sezioni, capitoli); rispetta la struttura logica ma chunk di dimensione molto variabile; (3) **chunking semantico** – il più sofisticato: con una finestra scorrevole, si seleziona una porzione di testo (es. 3 frasi), si divide in due parti, si calcola l’embedding di ciascuna, si calcola la distanza coseno tra i due embedding, e se la distanza supera una soglia si inserisce un confine di chunk (le due parti trattano argomenti diversi $\Rightarrow$ punto naturale di separazione). Questo trasforma il chunking da operazione sintattica a operazione **semantica**.

Metadata enrichment: arricchire ogni chunk con metadati (titolo, autore, data, sezione, parole chiave), usabili per filtrare risultati (“solo articoli dopo il 2020”), re-rankare (priorità a fonti autorevoli), fornire citazioni nella risposta generata.

7.8 Vector Database e Scalabilità

Per rendere il retrieval efficiente su milioni di documenti non si può calcolare la similarità con ogni chunk uno per uno (troppo lento) – servono i **vector database**: FAISS (Meta, open-source, CPU/GPU); Annoy (Spotify, random projection trees); HNSW (grafo navigabile gerarchico); Pinecone/Weaviate/Qdrant (soluzioni managed); ChromaDB (semplicità, integrazione rapida con LLM).

Approximate Nearest Neighbors (ANN). Invece di confrontare la query con tutti gli embedding, si crea una struttura che permette di “saltare” regioni dello spazio che sicuramente non contengono i vicini più prossimi. Es. HNSW costruisce un grafo multi-livello: livelli superiori = navigazione grossolana, salti grandi; livelli inferiori = navigazione fine, salti piccoli. Questo riduce la complessità da $O(n)$ (ricerca lineare) a $O(\log n)$, rendendo possibile il retrieval in real-time anche su dataset enormi.

7.9 Hybrid Search: Combinazione di Livelli Lessicale e Semantico

Combina retrieval lessicale (BM25) e semantico (embeddings), sfruttando il fatto che i due approcci hanno punti di forza complementari: BM25 eccellente per keyword esatte, nomi propri, codici, date; semantic search eccellente per concetti, sinonimi, parafrasi.

$$\text{score}(q, d) = \alpha \cdot \text{BM25}(q, d) + (1 - \alpha) \cdot \text{similarity}(\vec{q}, \vec{d})$$

dove $\alpha \in [0, 1]$ bilancia i due contributi. Alcuni sistemi usano approcci più sofisticati come il **re-ranking**: prima si recuperano molti candidati con BM25 (veloce), poi si re-rankano con embeddings (più accurati).

7.10 RAG come Integrazione di Paradigmi

La RAG è interessante non solo come tecnica, ma come punto di convergenza tra **tre grandi paradigmi** della NLP: **semantica distribuzionale** (rappresentazione vettoriale del significato), **information retrieval** (selezione di informazione rilevante), **modelli generativi** (produzione di linguaggio naturale). Pipeline: il significato viene **rappresentato** (embeddings), **cercato** (retrieval), **espresso** (generazione) – una delle architetture più complete del panorama attuale.

7.11 Evaluation: Misurare la Qualità di un Sistema RAG

Metriche per il retrieval: Precision@k, Recall@k, **MRR** (Mean Reciprocal Rank, posizione media del primo documento rilevante), **NDCG** (Normalized Discounted Cumulative Gain, tiene conto dell'ordine e della rilevanza graduata). **Metriche per la generazione**: **Faithfulness** (la risposta è supportata dai documenti recuperati?), **Answer Relevance** (la risposta è pertinente alla query?), **Context Relevance** (i documenti recuperati sono pertinenti?) – framework come **RAGAS** forniscono metriche automatiche per questi aspetti. **End-to-end evaluation**: dataset annotati con (query, risposta corretta, documenti rilevanti); metriche come BLEU/ROUGE (limitate: focus su n-gram overlap); valutazione umana (accuracy, completezza, coerenza); **LLM-as-a-judge** (usare un LLM forte per valutare le risposte).

7.12 Limiti e Questioni Aperte

Nonostante i vantaggi, la RAG non elimina le difficoltà fondamentali. Il problema principale resta: *il modello non capisce documenti. Li utilizza*. Il retrieval introduce grounding, ma non garantisce comprensione semantica profonda.

Limiti tecnici: **dipendenza critica dalla qualità del retrieval** (se il retriever non trova documenti rilevanti, l'LLM non può generare una buona risposta – “garbage in, garbage out”); **difficoltà nel combinare informazioni da fonti multiple** (se la risposta richiede sintesi di informazioni sparse in molti documenti, la RAG fatica); **limiti della finestra di contesto** (anche con 100k+ token, non possiamo includere intere biblioteche); **latenza** (il retrieval aggiunge overhead temporale); **contraddizioni tra fonti** (cosa fare se documenti recuperati si contraddicono?).

Problemi aperti: **reasoning multi-hop** (query che richiedono concatenare informazioni da step multipli); **temporal reasoning** (gestire informazioni che cambiano nel tempo); **bias e fairness** (i documenti possono contenere bias, che vengono amplificati); **attribution e citation** (come fornire citazioni precise e verificabili).

7.13 Direzioni Future

Adaptive RAG (il sistema decide autonomamente quando e come usare il retrieval); **Multimodal RAG** (integrare testo, immagini, tabelle, codice); **Conversational RAG** (gestire dialoghi multi-turno con memoria); **fine-tuning del retriever** (addestrare retriever e generatore end-to-end insieme); **knowledge editing** (aggiornare selettivamente la conoscenza del modello).

7.14 Conclusione

La RAG rappresenta una delle evoluzioni più significative nell’NLP recente. Non sostituisce i modelli generativi, ma li completa. Non elimina le allucinazioni, ma le riduce drasticamente. Non risolve il problema del significato, ma lo riformula in termini più pragmatici: *il significato non è solo ciò che un modello ha appreso, ma anche ciò che è in grado di recuperare e utilizzare nel momento giusto*. Questo sposta l’attenzione dalla rappresentazione statica del significato alla sua **attivazione dinamica e contestuale** – apre una nuova direzione per la ricerca: *non più modelli che sanno tutto, ma sistemi che sanno interagire intelligentemente con la conoscenza*.

7.15 Lab-5 (Dispense): Costruire un Sistema RAG su Articoli Scientifici NLP

Le dispense propongono un sistema RAG completo su MaartenGr/arxiv_nlp, per creare un assistente che risponde a domande sulla letteratura scientifica NLP citando le fonti. **Setup:** datasets, transformers, sentence-transformers, faiss-cpu, torch, accelerate. **Step 1:** caricamento dataset, auto-rilevamento colonne titolo/abstract, combinazione titolo+abstract in un unico chunk (semanticamente coesi), metadata separati per citazioni. **Step 2:** embeddings con all-MiniLM-L6-v2 (alternative: all-mpnet-base-v2 più accurato; SciBERT per domini scientifici). **Step 3:** vector store FAISS IndexFlatIP (ricerca esatta; alternative IndexIVFFlat per clustering+approssimata, IndexHNSW per il miglior rapporto velocità/accuratezza). **Step 4:** funzione di retrieval top-k. **Step 5:** LLM locale leggero (Qwen2.5-0.5B-Instruct, alternative SmolLM-360M, Phi-2). **Step 6:** funzione rag_answer end-to-end – retrieval → costruzione contesto (troncato a 1500 caratteri per modelli piccoli) → prompt con chat template → generazione (temperature 0.3) → risposta + sezione fonti.

Estensioni possibili: hybrid search (BM25+embeddings), re-ranking con cross-encoder, evaluation (Precision@k, Recall@k, faithfulness), Self-RAG, interfaccia (Gradio/Streamlit).

Takeaway principali delle dispense: (1) la qualità del retrieval è critica – un cattivo retriever rende inutile anche il miglior LLM; (2) il chunking richiede bilanciamento tra contesto e precisione; (3) l’hybrid search spesso supera gli approcci singoli; (4) i modelli

locali (Qwen, Phi) sono alternative valide a OpenAI; (5) la valutazione è multidimensionale (retrieval, faithfulness, relevance). **La RAG trasforma gli LLM da “oracoli statici” a “ricercatori dinamici”**, capaci di accedere, valutare e sintetizzare conoscenza esterna.

Nota sull’implementazione effettivamente svolta: il laboratorio concretamente realizzato (vedi Parte II, Sez. 16.1 e seguenti) usa scelte leggermente diverse da quelle suggerite nelle dispense – encoder SciBERT (768-dim) invece di MiniLM, LLM Phi-3.5-mini-instruct invece di Qwen, e un confronto sistematico hybrid (BM25+FAISS+RRF) vs. baseline su 5 dimensioni di corpus (1K–44.9K documenti), non presente come tale nelle dispense. Sono scelte implementative equivalenti nello spirito del laboratorio proposto, riportate in dettaglio nella Parte II.

8 Large Language Models, Prompting e Human-AI Interaction

8.1 Introduzione

I **Large Language Model (LLM)** rappresentano un punto di svolta nell’NLP: a differenza dei modelli tradizionali, addestrati per task specifici, sono sistemi *generalisti* capaci di svolgere molteplici compiti attraverso il solo linguaggio naturale. Questa capacità emerge da due fattori: la **scala** (modelli con miliardi di parametri su enormi quantità di testo catturano pattern linguistici complessi e conoscenza implicita) e l’architettura **Transformer** (il meccanismo di attention cattura dipendenze a lungo raggio, superando i limiti delle reti ricorrenti).

Ciò che rende gli LLM particolarmente significativi non è solo la capacità tecnica, ma il modo in cui modificano l’interazione tra umani e sistemi di AI: per la prima volta possiamo comunicare con un sistema computazionale usando linguaggio naturale non vincolato, senza tradurre le intenzioni in comandi predefiniti o interfacce strutturate.

8.2 Dai Modelli Base agli Instruction-Tuned Models

I **base language model** sono addestrati esclusivamente sulla predizione del token successivo: completano testo in modo fluido ma **non seguono istruzioni**. Alla domanda “La capitale dell’Italia è”, un modello base produce una continuazione naturale (“Roma, situata nel Lazio...”), non una risposta. Gli **instruction-tuned model** sono ulteriormente addestrati su coppie domanda-risposta: interpretano l’input come una richiesta e generano risposte appropriate (“Qual è la capitale dell’Italia?” → “La capitale dell’Italia è Roma”).

Il passaggio da base a instruction-tuned avviene tramite: **Supervised Fine-Tuning (SFT)** – addestramento su migliaia di esempi annotati di interazioni umane di alta qualità (da solo insufficiente per preferenze umane complesse); **Reinforcement Learning from Human Feedback (RLHF)** – umani valutano risposte diverse, creando un modello

di ricompensa che cattura le preferenze, usato poi per ottimizzare il modello; **Direct Preference Optimization (DPO)** – tecnica più recente che ottiene risultati simili a RLHF senza la complessità del reinforcement learning esplicito.

8.3 Prompting: l'Arte di Comunicare con gli LLM

Il **prompting** è il processo di formulare input testuali che guidano il comportamento del modello – non “dare comandi”, ma costruire un’interfaccia linguistica efficace tra intenzione umana e capacità del modello.

Perché è cruciale: con gli LLM l’interfaccia è il linguaggio stesso, non pulsanti/menu/moduli predefiniti. Questo dà **flessibilità** senza precedenti (task arbitrariamente complessi senza attendere nuove funzionalità), ma introduce **ambiguità** intrinseca (la stessa richiesta può essere interpretata in modi diversi) e **imprevedibilità** (piccole variazioni del prompt → output molto diversi), richiedendo un approccio iterativo.

8.3.1 Principi Fondamentali

1. **Chiarezza e specificità:** un prompt vago produce output generici. “Parlami di Roma” vs. “Descrivi in tre paragrafi la storia di Roma repubblicana (509-27 a.C.), includendo organizzazione politica, espansione territoriale e fattori di transizione verso l’impero” – la seconda versione fornisce vincoli chiari.
2. **Delimitatori:** marcatori espliciti (virgolette triple, tag XML, separatori) per separare testo da analizzare e istruzioni su come analizzarlo, evitando che il modello confonda le due parti.
3. **Formato di output esplicito:** specificare JSON con campi precisi, numero di frasi/parole per un riassunto, ecc. – elimina ambiguità e rende l’output direttamente utilizzabile.
4. **Iterazione:** raramente il primo tentativo è ottimale. Ciclo: generare → analizzare l’output → identificare problemi (ambiguità, info mancanti, formato errato) → raffinare → ripetere.

8.3.2 Strategie Avanzate

Chain-of-Thought (CoT): chiedere al modello di “pensare ad alta voce” migliora drasticamente le performance su task di ragionamento. “Ho 15 mele, ne mangio 3, poi compro il doppio di quelle mangiate. Quante mele ho ora? *Ragiona passo-passo*” → il modello esplicita: $15 - 3 = 12$, doppio di 3 = 6, $12 + 6 = 18$. Forzato a esplicitare i passaggi intermedi, il modello commette meno errori di calcolo e mantiene coerenza logica. Particolarmente efficace per matematica, logica, pianificazione multi-step.

Few-shot learning: fornire 1-5 esempi espliciti di input-output nel prompt, invece di descrivere solo verbalmente il task (zero-shot). Per sentiment analysis: “Ottimo prodotto, lo consiglio! → Positivo”, “Non funziona, deluso → Negativo”, poi il caso da classificare – gli esempi forniscono un template concreto.

Self-Consistency: si generano multiple soluzioni indipendenti (temperature più alta) e si seleziona la risposta più frequente (majority voting) – migliora l’accuratezza su task con soluzioni ben definite.

Role Prompting: assegnare un ruolo (“Sei un esperto di storia romana”, “Sei un programmatore senior Python”) influenza positivamente tono, livello di dettaglio e competenza apparente della risposta.

Cosa evitare: negazioni eccessive – specificare positivamente cosa il modello *deve* fare (“Usa linguaggio semplice per un bambino di 10 anni”) è più efficace che dire cosa *non* deve fare (“Non usare termini tecnici”); **ambiguità referenziale** – pronomi vaghi (“Riassumilo”) creano incertezza, meglio essere espliciti (“Riassumi il testo sopra in 3 bullet points”).

8.4 Task Comuni con gli LLM

Task	Caratteristiche principali
Summarization	Può essere orientata (per aspetto specifico, livello di dettaglio, audience); <i>sintesi progressiva</i> – da paragrafo a 3 frasi a 1 frase a parola chiave.
Inference/Classificazione	Sentiment analysis, NER, topic detection definiti interamente a prompt, con categorie arbitrarie on-the-fly senza training di modelli dedicati.
Trasformazione	Traduzione (anche tra registri/stili), conversione di formato (CSV→JSON), correzione di errori e normalizzazione.
Espansione	Inverso della sintesi: da bullet points a email, da outline ad articolo completo. Introduce il parametro temperatura .

Temperatura: controlla la casualità/“creatività” del modello. **Bassa** (0.0–0.3): output deterministico, ideale per documentazione tecnica, codice, risposte fattuali. **Media** (0.5–0.7): bilancia prevedibilità e varietà. **Alta** (0.8–1.0+): output vario e imprevedibile, utile per scrittura creativa e brainstorming. La scelta dipende dal contesto: temperatura bassissima per un contratto legale, alta per una campagna marketing creativa.

8.5 Limiti e Problematiche degli LLM

Allucinazioni: il problema più insidioso. Alla domanda “Chi ha vinto il Nobel per la letteratura nel 2025?”, un LLM potrebbe rispondere con sicurezza inventando uno scrittore e un romanzo inesistenti. Non è un bug occasionale ma una conseguenza diretta del funzionamento: gli LLM sono generatori statistici che producono ciò che è *probabile* seguire il prompt, non ciò che è *vero* – e non mostrano esitazione, generando fatti inventati con la stessa fluidità di fatti reali. Mitigazione principale: **RAG** (ancora il modello a documenti verificabili); nessuna tecnica elimina completamente il rischio.

Limite	Descrizione e mitigazione
Conoscenza statica	La conoscenza è “congelata” al training cutoff; non si aggiorna autonomamente. Mitigazione: retrieval esterno/strumenti di ricerca in tempo reale.
Bias e fairness	Riflette stereotipi presenti nei dati (professione-genere, rappresentazione culturale squilibrata). Mitigazione: curation del dataset, debiasing, red teaming, monitoraggio.
Ragionamento superficiale	Eccelle nel pattern matching, fatica con ragionamento causale profondo, pianificazione multi-step, comprensione fisica del mondo (non ha esperienza diretta).
Context window limitato	Performance degrada con contesti lunghi (“lost in the middle”); costi quadratici nella lunghezza del contesto.
Costi computazionali	Training di milioni di euro ed emissioni CO ₂ equiparabili a centinaia di voli intercontinentali; anche l’inference ha costi energetici non trascurabili.

8.6 Human-AI Interaction: la Nuova Frontiera

8.6.1 Dal Task-Oriented al Goal-Oriented

Le interfacce tradizionali sono **task-oriented**: l’utente deve conoscere in anticipo le funzionalità disponibili e adattare il proprio obiettivo alle capacità del sistema (menu, form, comandi predefiniti). Le interfacce LLM sono **goal-oriented**: l’utente esprime obiettivi in linguaggio naturale, raffina iterativamente attraverso il dialogo, scopre le capacità del sistema man mano che interagisce. Abbassa drasticamente la barriera d’ingresso, ma introduce una sfida: come sa l’utente cosa chiedere se non conosce i limiti del sistema?

8.6.2 Nuovi Pattern di Interazione

- **Co-creazione:** l’AI diventa partner creativo in un ciclo iterativo (umano fornisce idea/direzione → AI espande con dettagli/varianti → umano seleziona e dà feedback → si ripete). Si applica a scrittura, programmazione, design: l’AI non sostituisce l’umano, amplifica le sue capacità fornendo rapidamente alternative.

- **Debugging conversazionale:** invece di interpretare messaggi di errore criptici, l'utente dice "L'output non è quello che volevo, voglio che sia più conciso"; il sistema chiede chiarimenti o propone alternative – abbassa il carico cognitivo, l'utente non deve tradurre il problema in termini tecnici.
- **Delega progressiva:** livelli crescenti di autonomia, da task ben definiti ("Traduci questo testo") a vincoli/preferenze ("mantenendo il tono formale") a obiettivi generali ("Adatta per un pubblico internazionale non tecnico") – il modello deve inferire sempre più informazioni da solo, richiedendo fiducia crescente.

8.6.3 Sfide nella Human-AI Interaction

Calibrazione della fiducia: gli utenti si polarizzano. **Over-trust:** accettano output senza verifica, delegano decisioni critiche senza controllo – pericoloso perché gli LLM presentano errori con la stessa sicurezza di risposte corrette. **Under-trust:** ignorano suggerimenti validi, verificano ossessivamente, sotto-utilizzano il sistema. Il problema di fondo: gli LLM non comunicano bene la propria incertezza (non dicono "forse"). Soluzioni possibili (imperfette): confidence scores, evidenziazione di parti incerte, presentazione di interpretazioni alternative.

- **Responsabilità e accountability:** quando un LLM genera contenuto problematico, chi è responsabile – l'utente, il creatore del modello, l'organizzazione che lo ha deployato? A differenza del software tradizionale (dove si può risalire a una riga di codice specifica), il comportamento emerge da pattern statistici su miliardi di esempi: non c'è una singola causa identificabile. Domande aperte su trasparenza, audit, meccanismi di appello.
- **Opacità del sistema:** gli LLM sono **black box** – l'utente non sa perché il modello ha dato quella risposta, su quali dati è stato addestrato, come funziona internamente. Rende difficile debuggare, fidarsi, imparare. Possibili mitigazioni: explainability, transparency reports, audit trails.
- **Deskilling:** rischio che gli utenti perdano competenze fondamentali delegando troppo all'AI (scrittura, programmazione, pensiero critico) – fenomeno non nuovo (calcolatrici, GPS, correttori ortografici), ma gli LLM operano su processi di pensiero e creatività, non solo calcoli o memoria. Bilanciamento ideale: l'AI come **amplificatore** di capacità esistenti, non sostituto.

8.6.4 Design Principles per Interfacce LLM

1. **Rendere l'incertezza visibile:** confidence scores, evidenziazione di parti incerte, presentazione esplicita di interpretazioni alternative.
2. **Supportare l'iterazione:** bottoni per raffinare ("più dettaglio", "più conciso", "spiega il ragionamento"), storia delle varianti, undo/redo accessibili – l'interazione non deve mai essere un vicolo cieco.
3. **Educare l'utente:** spiegare attivamente capacità e limiti del sistema, fornire esempi di prompt efficaci, mostrare best practice attraverso l'uso.

4. **Dare controllo:** parametri (temperatura, lunghezza, dettaglio) regolabili con controlli intuitivi, possibilità di intervento manuale, modalità “explain why” per il ragionamento del modello.

8.6.5 Casi d’Uso Emergenti

Assistenti personali contestuali altamente specializzati per dominio (medico, legale, ricerca) sia nella conoscenza sia nello stile di interazione. **Cognitive scaffolding:** l’AI non produce output finali ma facilita il pensiero – genera idee in brainstorming, fa da “devil’s advocate” sfidando assunzioni, pone domande socratiche invece di dare risposte dirette (da “rispositore” a “facilitatore di pensiero”). **Mediazione culturale e linguistica:** oltre la traduzione letterale – adatta riferimenti culturali, ricrea umorismo basato su giochi di parole, localizza espressioni idiomatiche, spiega contesti culturali opachi.

Laboratorio opzionale (dalle dispense): un approfondimento non obbligatorio propone 6 esperimenti pratici con modelli piccoli (Qwen2.5-0.5B-Instruct, Phi-3.5-mini-instruct, eseguibili su CPU) per osservare empiricamente: l’effetto del CoT su problemi aritmetici multi-step; la consistenza del few-shot prompting sulla classificazione; le allucinazioni su domande oltre il training cutoff o su entità inesistenti (e la mitigazione tramite richiesta esplicita di incertezza); l’effetto della temperatura sulla varietà/coerenza dell’output; la sensibilità a piccole riformulazioni del prompt e all’ordine degli esempi few-shot; i limiti del ragionamento controfattuale e dei sillogismi senza CoT. Per ciascun esperimento si tracciano accuratezza, consistenza tra run, latenza e failure mode – un esercizio di falsificazione empirica dei limiti discussi in questo capitolo, non di implementazione di un sistema.

8.7 Conclusioni

Takeaway chiave: (1) il prompting è un’abilità che richiede pratica e iterazione; (2) gli LLM hanno limiti strutturali (allucinazioni, bias, ragionamento superficiale); (3) le strategie avanzate (CoT, few-shot, self-consistency) migliorano significativamente le performance; (4) la human-AI interaction richiede nuovi principi di design; (5) la responsabilità nell’uso degli LLM è cruciale.

Direzioni future: multimodalità (integrazione testo-immagine-audio); agenticità (LLM come agenti autonomi con strumenti); personalizzazione (modelli adattati a utenti/domini); efficienza (modelli più piccoli con capacità maggiori); alignment (modelli sempre più allineati con valori umani).

Messaggio finale: gli LLM non sono “intelligenza artificiale generale”. Sono sistemi potenti ma limitati, che richiedono comprensione dei meccanismi, uso consapevole e critico, verifica degli output, considerazione etica. La vera potenza degli LLM emerge quando vengono usati come **amplificatori dell’intelligenza umana**, non come sostituti.

9 Valutazione per la Terza Parte del Corso di TLN

9.1 La Filosofia di Valutazione

Nelle parole del Prof. Di Caro: *“L’obiettivo non è superare l’esame. L’obiettivo è diventare qualcuno che sa fare NLP – che sa quando usare BERT e quando bastano SVM, che capisce perché un topic model ha fallito, che può spiegare trade-off complessi a non-tecnici, che costruisce sistemi robusti invece di demo fragili.”* L’esame è strutturato per testare questo tipo di competenza profonda, non la memorizzazione superficiale.

9.2 Struttura dell’Esame

L’esame è composto da due parti complementari: **Parte Teorico-Critica** (domande sulla parte descrittiva e concettuale del corso) e **Parte Laboratoriale** (discussione degli esercizi pratici implementati: Lab-*n* più il progetto esteso).

9.3 Parte I: Teoria e Pensiero Critico

Tre criteri guidano la valutazione, in ordine di importanza crescente:

1. **Linguaggio tecnico preciso:** capacità di usare terminologia tecnica con precisione, senza vaghe approssimazioni. “Embedding” non è sinonimo generico di “rappresentazione”; “attention” non significa “il modello guarda qualcosa”; TF-IDF non è “una specie di frequenza delle parole”. *La precisione lessicale riflette precisione concettuale.*
2. **Chiarezza concettuale e contestuale:** idee precise sulle definizioni e comprensione del contesto storico, metodologico e applicativo. Sapere “cosa è Word2Vec” senza capire *perché* fu rivoluzionario nel 2013, quali problemi risolveva rispetto al bag-of-words, quali limiti BERT ha poi superato, è memorizzazione, non comprensione.
3. **Pensiero comparativo e integrativo:** capacità di mettere in dialogo concetti diversi, confrontare approcci, valutare trade-off, costruire ponti tra aree del corso. È la competenza che distingue un esperto da un novizio: non solo definire un embedding, ma sapere quando Word2Vec basta e quando serve BERT contestuale, quando TF-IDF batte ancora gli embeddings neurali su certi task di retrieval, quali assunzioni porta ciascun metodo e come limitano l’applicabilità in scenari reali.

9.4 Parte II: Laboratorio e Implementazione

La seconda parte parte dagli esercizi di laboratorio implementati, ma non richiede di recitare il codice a memoria: richiede di dimostrare **ownership** – che il codice sia compreso in cosa fa, perché lo fa così, e cosa si potrebbe fare diversamente.

1. **Conoscenza personale del codice:** capacità di navigare il codice, spiegare sezioni specifiche, modificare al volo, rispondere a “cosa succederebbe se...”. Esempi di domande: “Qui usi `n_components=5` in UMAP. Cosa succederebbe con 15? Quando vorresti più componenti?”; “Come hai scelto `min_cluster_size=50`? Hai sperimentato con altri valori?”

2. **Correttezza, semplicità, leggibilità:** codice che funziona ma anche leggibile, con nomi variabili sensati, struttura chiara, commenti dove servono (e silenzio dove sono ovvi) – nel mondo reale il codice è letto molto più spesso di quanto è scritto.
3. **Profondità di analisi dei risultati:** il criterio che separa competenza superficiale da profonda. Non basta “ho ottenuto 15 topic”: bisogna saper dire “3 sembrano overlap semantico, 2 sono rumore probabilmente, il topic 7 è sorprendentemente coerente e cattura X che non mi aspettavo; ho provato `min_cluster_size=100` e sono scesi a 10 topic più stabili ma ho perso granularità su Y; il trade-off dipende da...”. È il cuore del metodo scientifico: riflettere criticamente sui risultati, inclusi quelli inaspettati o i fallimenti.

9.5 Griglia di Valutazione Sintetica

Parte I – Teoria (circa 50% del voto)

Criterio	Insufficiente	Eccellente
Linguaggio tecnico	Termini vaghi, imprecisi o usati scorrettamente	Terminologia precisa, usata con consapevolezza di sfumature
Chiarezza concettuale	Definizioni isolate, senza contesto storico/metodologico	Concetti contestualizzati, con motivazioni, assunzioni, limiti
Pensiero comparativo	Risposte isolate per ogni domanda, nessuna connessione	Capacità di confrontare, integrare, valutare trade-off

Parte II – Laboratorio (circa 50% del voto)

Criterio	Insufficiente	Eccellente
Ownership codice	Non sa spiegare sezioni, “ho copiato da un tutorial”	Naviga fluentemente, spiega scelte, modifica al volo
Qualità codice	Funziona ma illeggibile, nomi variabili opachi	Leggibile, strutturato, commenti appropriati
Analisi risultati	“Ho ottenuto X topic” senza riflessione	Validazione qualitativa, comparazione configurazioni, consapevolezza dei limiti

10 Conclusioni: Semantica, Conoscenza e Futuro del NLP

10.1 Il Percorso Compiuto

Il corso ha tracciato un percorso attraverso diverse prospettive sulla comprensione computazionale del linguaggio – non una sequenza di tecniche, ma un viaggio concettuale su come catturare, rappresentare e generare significato.

- **Semantica computazionale** (Cap. 1): il significato come inventario (lessicale), come condizioni di verità (formale), come distribuzione nell'uso (distribuzionale) – complementari, non alternativi; i sistemi moderni, inclusi gli LLM, combinano implicitamente tutti e tre.
- **Definizioni** (Cap. 2): ponte tra conoscenza esplicita e computazione; la distinzione semasiologico/onomasiologico mostra che la direzione del processo semantico non è neutrale – rilevante quando si integra retrieval con generazione (RAG).
- **Privacy e misinformation** (Cap. 3): problemi semantici tanto quanto computazionali; l'explainability è una necessità etica, non un lusso (progetto KURAMi).
- **Polisemia** (Cap. 4): l'ambiguità non è un bug ma una feature del linguaggio; la complessità polisemica non è universale ma varia per lingua, cultura, dominio.
- **Basic level** (Cap. 5): esiste una granularità cognitivamente privilegiata, né troppo generica né troppo specifica, ma non universale – dipende da contesto, competenza, dominio.
- **Pre-LLM NLP** (Cap. 6): da simbolico a statistico a neurale a Transformer – non successione che cancella, ma **stratificazione**: BM25, SVM, TF-IDF restano in produzione non per ignoranza ma per scelta (interpretabilità, costo, latenza).
- **RAG** (Cap. 7): punto di convergenza tra retrieval classico e generazione neurale; non risolve il problema del significato, lo riformula – non solo ciò che un modello ha appreso, ma ciò che è in grado di recuperare e usare al momento giusto.
- **LLM e Human-AI Interaction** (Cap. 8): capacità impressionanti, limiti strutturali, e soprattutto una trasformazione di come umani e AI interagiscono – da interfacce rigide a conversazioni aperte, con benefici e rischi.

10.2 Temi Trasversali

Tensione	Sintesi
Simbolico vs. subsimbolico	Rappresentazioni esplicite (interpretabili, rigide) vs. distribuite (potenti, opache). Direzione promettente: neuro-symbolic AI (es. RAG combina retrieval esplicito + generazione neurale).
Conoscenza implicita vs. esplicita	Parametri LLM (impliciti, non verificabili) vs. knowledge base tradizionali (esplicite, statiche, costose). Direzione: sistemi ibridi che si aggiornano/verificano a vicenda.
Task-specific vs. general-purpose	Pre-LLM: un modello per task; LLM: un modello, prompting multiplo. Tendenza attuale: LLM come backbone, fine-tuned/augmented per task specifici.
Dati vs. modelli	Risposta in evoluzione – dati (era statistica), architettura (era deep learning), scala di entrambi (era LLM); dati e modelli co-evolvono.
Evaluation	Nessuna metrica perfetta (BLEU/ROUGE correlano male con qualità percepita; valutazioni umane costose/soggettive); serve triangolazione di metodi.
Multilingualità	L’NLP ha storicamente favorito l’inglese; concetti “universali” (basic level, polisemia) variano cross-linguisticamente in modi sottili.

10.3 Sfide Aperte

Comprensione vs. pattern matching: gli LLM completano frasi fluentemente, ma capiscono la causalità? Hanno un modello mentale del codice che generano? Il dibattito su cosa significhi “comprensione” è filosofico quanto tecnico, ma ha implicazioni pratiche: se un modello non comprende veramente, i suoi errori saranno sistematici e imprevedibili.

Groundedness: il linguaggio umano è *grounded* nel mondo fisico/sociale; gli LLM sono addestrati solo su testo, senza esperienza diretta – i sistemi multimodali affrontano questa limitazione, ma siamo lontani da una vera *grounded understanding*. **Composizionalità:** i Transformer processano sequenze, non alberi sintattici o grafi semantici; il CoT aiuta ma non risolve il problema strutturale. **Long-term memory:** nessuna memoria episodica a lungo termine – ogni interazione è isolata, mitigata solo da memoria esterna (RAG). **Efficienza e sostenibilità:** costi economici/ambientali del training e dell’inference, e questioni di accesso equo (chi può permettersi di addestrare questi modelli?). **Robustezza:** vulnerabilità ad attacchi adversariali (prompt injection, jailbreak) in una corsa continua tra attaccanti e difensori. **Alignment:** RLHF allinea a quali valori, definiti da chi? Non esistono valori universali, sfida profondamente interdisciplinare.

10.4 Direzioni Future

Agenticità e tool use (LLM come agenti che eseguono codice, interrogano database, navigano il web – nuove capacità ma anche nuovi rischi); multimodalità profonda (rappresentazioni unificate testo-visione-audio-tatto, più vicine alla cognizione *embodied*); personalizzazione e contestualizzazione (modelli che si adattano a utenti specifici, con questioni di privacy e lock-in); neuro-symbolic integration (knowledge graph + embeddings, reasoning logico guidato da LLM); modelli più piccoli ed efficienti (distillation, quantization, mixture-of-experts – Phi-2/Mistral competono con modelli 10× più grandi); evaluation next-generation (benchmark dinamici oltre GLUE/MMLU, che testino reasoning causale, planning, robustezza).

10.5 Riflessioni Metodologiche

Oltre ai contenuti tecnici, il corso ha cercato di trasmettere un approccio metodologico: **pensiero critico sui sistemi** (capire assunzioni, limiti, trade-off, non solo l'uso di un tool); **iterazione e sperimentazione** (l'NLP è empirico – design sperimentale attento, riproducibilità); **interdisciplinarietà** (linguistica, psicologia cognitiva, filosofia del linguaggio, sociologia, etica); **consapevolezza storica** (gli embeddings distribuiti echeggiano i vector space model anni '70, il prompting ricorda i template-based system – conoscere la storia evita di ripetere errori); **responsabilità etica** (le scelte su dati, valutazione, applicazioni hanno conseguenze reali).

10.6 Il Futuro è Conversazione

Se dovessimo riassumere in una frase la trasformazione portata dagli LLM: *abbiamo spostato l'interfaccia human-computer dal codice alla conversazione*. Prima degli LLM, interagire con un computer richiedeva tradurre intenzioni in linguaggio formale (command-line, SQL, API, GUI); l'utente si adattava alla macchina. Con gli LLM, la macchina si adatta all'utente: obiettivi in linguaggio naturale, raffinamento tramite dialogo, scoperta delle capacità conversando – con limiti e rischi. Questo abbassa drasticamente le barriere d'accesso, ma introduce responsabilità: se tutti possono usare l'AI, come si previene il *misuse*? Se l'AI è così fluente, come distinguiamo verità da finzione? Se deleghiamo tanto, cosa perdiamo in competenza? La conversazione come interfaccia non è solo più comoda: è qualitativamente diversa – cambia cosa è possibile, chi può partecipare, come pensiamo al rapporto human-machine. Siamo solo all'inizio dell'esplorazione di queste implicazioni.

Parte II

Laboratori (Dettaglio Implementativo e Risultati)

11 Introduzione: Contenuto vs. Forma Lessicale

Due direzioni nel lessico:

- **Semasiologica** (Forma $\rightarrow$ Contenuto): dato un lemma, cerco il suo significato. Direzione classica del dizionario.
- **Onomasiologica** (Contenuto $\rightarrow$ Forma): data una descrizione/definizione, trovo la parola corretta. Problema di *content-to-form retrieval*.

Il percorso del corso copre tre macro-temi:

1. **Similarità lessicale/semantica** (Lab 1–2): misure Jaccard e TF-IDF coseno su definizioni umane
2. **Topic Modeling neurale** (Lab 4): pipeline BERTopic per scoperta automatica di argomenti
3. **RAG ibrido** (Lab 5): retrieval aumentato con FAISS semantico + BM25 lessicale + RRF + LLM

12 I Quattro Tipi Concettuali

Nei lab 1 e 2 si lavora su quattro concetti scelti lungo due assi: **concretezza** (concreto/astratto) e **specificità** (generico/specifico).

Tipo	Sigla	Esempio	Concretezza	Specificità
Concrete Generic	CG	Albero (Tree)	Concreto	Generico
Concrete Specific	CS	Teiera (Teapot)	Concreto	Specifico
Abstract Generic	AG	Musica (Music)	Astratto	Generico
Abstract Specific	AS	Etica (Ethics)	Astratto	Specifico

Caratteristiche per tipo:

- **CG (Albero)**: referente fisico, classe ampia. Definizioni usano vocabolario condiviso (radici, tronco, rami, foglie). Alta coerenza cross-individuale.
- **CS (Teiera)**: referente fisico specifico, funzione primaria. Vocabolario concentrato su funzione (infusione, versare).
- **AG (Musica)**: nessun referente fisico, classe ampia. Definizioni variano enormemente: operative, sensoriali, emotive.
- **AS (Etica)**: nessun referente fisico, dominio filosofico. Alta influenza culturale e contestuale. Eterogeneità massima.

13 Similarità Lessicale e Semantica (Lab 1)

13.1 SimLex – Indice di Jaccard

$$\text{SimLex}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

dove A e B sono insiemi di parole lemmatizzate e filtrate (stopwords rimosse) delle due definizioni.

Cosa misura: sovrapposizione *lessicale* pura – due testi con stesso significato ma parole diverse → score basso.

Pipeline di preprocessing:

1. Lowercase
2. Rimozione punteggiatura
3. Tokenizzazione (NLTK `word_tokenize`)
4. Rimozione stopwords (NLTK `stopwords.words('italian')`)
5. Lemmatizzazione
6. Conversione in `set`

```

1 from nltk.corpus import stopwords
2 from nltk.tokenize import word_tokenize
3 import re, string
4
5 def preprocess_to_set(text, lang='italian'):
6     text = text.lower()
7     text = re.sub(r'[^\w\s]', '', text)
8     tokens = word_tokenize(text, language=lang)
9     stop = set(stopwords.words(lang))
10    tokens = [t for t in tokens if t not in stop and len(t) > 1]
11    return set(tokens) # set per Jaccard
12
13 def simlex(defA, defB):
14     A = preprocess_to_set(defA)
15     B = preprocess_to_set(defB)
16     if not A and not B:
17         return 0.0
18     return len(A & B) / len(A | B)

```

13.2 SimSem – Coseno TF-IDF

$$\text{SimSem}(A, B) = \cos(\vec{v}_A, \vec{v}_B) = \frac{\vec{v}_A \cdot \vec{v}_B}{\|\vec{v}_A\| \cdot \|\vec{v}_B\|}$$

dove $\vec{v}$ è la rappresentazione TF-IDF del testo.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{\text{df}(t)}$$

Cosa misura: prossimità *semantica* – cattura sinonimi e termini correlati che il semplice overlap non coglie.

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 def simsem(defA, defB):
5     corpus = [preprocess_text(defA), preprocess_text(defB)]
6     vectorizer = TfidfVectorizer()
7     tfidf_matrix = vectorizer.fit_transform(corpus)
8     return cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])
9     [0][0]

```

13.3 Confronto tra coppie – tutte le combinazioni

Il codice usa `itertools.combinations` per calcolare similarità tra tutte le coppie di concetti:

```

1 import itertools, csv
2
3 concetti = ['Musica', 'Etica', 'Albero', 'Teiera']
4 definizioni = {c: [...] for c in concetti} # lista def per c
5
6 risultati = []
7 for c1, c2 in itertools.combinations(concetti, 2):
8     for def1, def2 in zip(definizioni[c1], definizioni[c2]):
9         sl = simlex(def1, def2)
10        ss = simsem(def1, def2)
11        risultati.append({'concetto1': c1, 'concetto2': c2,
12                          'simlex': sl, 'simsem': ss})
13
14 # salva su CSV
15 with open('risultati_similarita.csv', 'w') as f:
16     writer = csv.DictWriter(f, fieldnames=risultati[0].keys())
17     writer.writeheader()
18     writer.writerows(risultati)

```


13.4 Risultati Lab 1

Concetto	SimLex (Jaccard)	SimSem (Coseno)
Albero (CG)	0.084	0.077
Teiera (CS)	0.057	0.062
Etica (AS)	0.034	0.041
Musica (AG)	0.034	0.036

Osservazioni chiave:

- Valori molto bassi ovunque (3–8%): enorme variabilità del linguaggio naturale anche per lo stesso referente.
- **Concreto > Astratto**: i concetti fisici hanno referenti che vincolano il vocabolario.
- **Per i concetti astratti, SimSem > SimLex**: il modello vettoriale allinea sinonimi e termini correlati che la Jaccard pura non vede. Questo dimostra il valore aggiunto della rappresentazione semantica.
- Etica: sebbene `simlex` = `simsem` = 0.034, le definizioni condividono *struttura concettuale* nonostante parole diverse.

14 Content-to-Form: Ricerca Onomasiologica (Lab 2)

14.1 Il Task

Ricerca onomasiologica: data una definizione in linguaggio naturale, recuperare il *synset* WordNet corretto.

Input: *"Branch of philosophy that studies human behavior by trying to define what is right and wrong"*

Output: `ethics.n.01`

È l'inverso della classica consultazione dizionario: si parte dal concetto/significato per trovare la forma lessicale.

14.2 Procedura Lab 2

1. **Selezione candidati**: per ogni concetto target, 5 synset WordNet candidati con relative glosse (incluse forme alternate e iperonimi)
2. **Preprocessing**: lowercase, rimozione punteggiatura, filtro stopwords su definizione utente E glosse
3. **Vettorizzazione TF-IDF**: l'intero corpus = [definizione_pulita] + [glosse_candidate_pulite]
4. **Coseno**: similarità tra definizione e ogni glossa candidata

5. **Predizione:** synset con score massimo = risposta del sistema
6. **Valutazione:** confronto con synset target

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 # Per ogni concetto: definizione umana + 5 glosse candidate
5 concetti = {
6     'Albero': {
7         'target': 'tree.n.01',
8         'definizioni': [...], # definizioni degli studenti
9         'candidati': {
10             'tree.n.01': 'a tall perennial woody plant having...',
11             'plant.n.01': 'a living organism lacking the power...',
12             # ... altri 3 candidati
13         }
14     },
15     # ... altri concetti
16 }
17
18 def content_to_form(defn, glosse_dict):
19     glosse = list(glosse_dict.values())
20     synsets = list(glosse_dict.keys())
21     corpus = [clean(defn)] + [clean(g) for g in glosse]
22
23     vec = TfidfVectorizer()
24     mat = vec.fit_transform(corpus)
25     scores = cosine_similarity(mat[0:1], mat[1:])[0]
26     best_idx = scores.argmax()
27     return synsets[best_idx], scores[best_idx]

```

14.3 Struttura Genus-Differentia

Schema Aristotelico: la migliore strategia definitoria è *genus proximum + differentia specifica*:

- **Genus:** categoria sovraordinata (spesso corrisponde all'ipernimo WordNet)
- **Differentia:** proprietà che distinguono il concetto dagli altri della stessa classe

Esempio efficace: "Branch of philosophy [genus] that studies moral values [differentia]"

Corrisponde quasi letteralmente alla glossa `ethics.n.01`: "the philosophical study of moral values and rules"

14.4 Risultati Lab 2

Concetto	Tipo	Accuracy	Score Avg	Score Max
Albero (CG)	Concreto/Generico	89.3%	0.1025	0.2919
Teiera (CS)	Concreto/Specifico	71.4%	0.1272	0.3063
Etica (AS)	Astratto/Specifico	42.9%	0.1524	0.6295
Musica (AG)	Astratto/Generico	32.1%	0.1298	0.3981

Aggregato per dimensione:

Dimensione	Accuracy	Score Avg
Concreto	80.4%	0.1149
Astratto	37.5%	0.1411
Generico	60.7%	0.1162
Specifico	57.1%	0.1398

Analisi:

- Concetti concreti: $2.1\times$ più recuperabili degli astratti. Il referente fisico vincola il vocabolario.
- Definizione migliore per Etica (score 0.63): “Branch of philosophy that studies human behavior by trying to define what is right and what is wrong, often influenced by historical and sociocultural context.” – ha genus-differentia esplicita.
- TF-IDF fallisce su sinonimia: richiede embedding contestuali per i concetti astratti.

15 Topic Modeling con BERTopic (Lab 4)

15.1 Architettura della Pipeline

Pipeline BERTopic (4 stadi):

Documenti $\xrightarrow{1. \text{ Embedding}}$ $\xrightarrow{2. \text{ UMAP}}$ $\xrightarrow{3. \text{ HDBSCAN}}$ $\xrightarrow{4. \text{ c-TF-IDF}}$ Topic interpretabili

15.2 Stadio 1: Embedding

Ogni documento viene rappresentato come vettore denso semantico tramite un *Sentence-Transformer*.

Modello	Dimensioni	Caratteristiche
thenlper/gte-small	384	Qualità alta, lento su CPU (13.484s)
all-MiniLM-L6-v2	384	Compatto, veloce su CPU, comparabile

15.3 Stadio 2: UMAP (Riduzione Dimensionale)

UMAP (Uniform Manifold Approximation and Projection): riduce le dimensioni preservando sia struttura locale che globale.

- **n_components**: dimensioni output (meno = più veloce, meno dettaglio semantico)
- **n_neighbors**: dimensione del vicinato locale (basso = focus locale; alto = focus globale)
- **min_dist**: distanza minima tra punti (0.0 per clustering; più alto per visualizzazione)
- **metric**: “cosine” per embedding normalizzati

Vantaggio su t-SNE: preserva struttura globale oltre a quella locale.

15.4 Stadio 3: HDBSCAN (Clustering)

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise):

- Identifica cluster di forma arbitraria (non solo sferici)
- Assegna automaticamente outlier all’etichetta -1
- **Non richiede** di specificare il numero di cluster a priori
- Parametro chiave: **min_cluster_size** – dimensione minima cluster

15.5 Stadio 4: c-TF-IDF (Estrazione Keywords Topic)

$$c\text{-TF-IDF}(t, c) = \frac{\text{freq}(t, c)}{|c|} \times \log \frac{N_{\text{docs}}}{|\{d : t \in d\}|}$$

dove c è il cluster (topic), non il singolo documento.

Differenza da TF-IDF classico: la frequenza viene calcolata *all’interno del cluster* come contesto, non del singolo documento. Produce keyword interpretabili e specifiche per ogni topic.

15.6 Configurazioni Testate

	Modello	UMAP	HDBSCAN	Effetto
Config 1 (base)	gte-small	comp=5, neigh=15	mcs=50	Bilanciato
Config 2 (fine)	MiniLM-L6	comp=10, neigh=10	mcs=30	Granulare
Config 3 (coarse)	MiniLM-L6	comp=5, neigh=30	mcs=150	Grossolano (fallimento)

15.7 Risultati Comparativi

Metrica	Config 1	Config 2	Config 3
# Topic (escl. -1)	151	171	49
Outlier count	14.357	12.747	14.899
Outlier %	31.9%	28.4%	33.1%
Runtime	~3h 47m	~17m 30s	~11m
Topic 0 keywords	speech, asr, recognition	dialogue, response, conv.	dialogue, the, to, and

Analisi failure mode Config 3:

- `min_cluster_size=150`: cluster troppo grandi e tematicamente eterogenei
- Il c-TF-IDF non riesce a estrarre segnale discriminativo: le stopwords emergono nel top-5
- Solo 49 topic (vs. 151–171 degli altri) con outlier percentuale paradossalmente più alta
- **Lezione**: parametri troppo grossolani → perdita di interpretabilità

Config 2 (raccomandata per CPU):

```

1 from sentence_transformers import SentenceTransformer
2 from bertopic import BERTopic
3 from umap import UMAP
4 from hdbscan import HDBSCAN
5
6 embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
7 umap_model = UMAP(n_components=10, n_neighbors=10,
8                   min_dist=0.0, metric='cosine')
9 hdbscan_model = HDBSCAN(min_cluster_size=30,
10                          cluster_selection_method='eom',
11                          prediction_data=True)
12
13 topic_model = BERTopic(
14     embedding_model=embedding_model,
15     umap_model=umap_model,
16     hdbscan_model=hdbscan_model,
17     language="english",
18     verbose=True
19 )
20
21 topics, probs = topic_model.fit_transform(docs)
22 print(topic_model.get_topic_info().head(10))

```

Dataset: MaartenGr/arxiv_nlp (HuggingFace) – 44.949 abstract di paper NLP.

16 RAG: Retrieval-Augmented Generation

16.1 Architettura Generale

RAG combina un *retriever* (recupera documenti rilevanti) con un *generatore* (LLM che risponde basandosi sul contesto):

$$\text{Query} \rightarrow \underbrace{[\text{Retriever}]}_{\text{top-}k \text{ docs}} \rightarrow \underbrace{[\text{Context Assembly}]}_{\text{concatenazione}} \rightarrow \underbrace{[\text{LLM}]}_{\text{risposta}} \rightarrow \text{Answer}$$

Vantaggi rispetto a LLM standalone:

- Accesso a conoscenza esterna e aggiornabile senza retraining
- Riduzione delle *allucinazioni* (risposta ancorata ai documenti)
- Possibilità di citare le fonti
- Adattabile a domini specifici

16.2 Retrieval Semantico con FAISS + SciBERT

FAISS (Facebook AI Similarity Search): libreria per nearest-neighbor efficiente su vettori densi.

- **IndexFlatIP**: inner product search su vettori normalizzati = cosine similarity
- Vettori: SciBERT (`allenai/scibert_scivocab_uncased`), 768 dimensioni
- SciBERT: BERT pre-trained su 1.14M paper scientifici (Semantic Scholar)

$$\text{Cosine}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \cdot \|\vec{d}\|} \quad (\text{equivalente a inner product su vettori normalizzati})$$

Punti di forza: cattura relazioni semantiche e sinonimi

Limiti: impreciso su termini tecnici esatti (acronimi, nomi di algoritmi)

16.3 Retrieval Lessicale con BM25

BM25 (Best Match 25):

$$\text{BM25}(D, Q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgl}}\right)}$$

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}$$

dove $k_1 = 1.5$, $b = 0.75$, N = dimensione corpus, $n(q_i)$ = doc frequency del termine.

- **Saturazione:** l'effetto della frequenza del termine raggiunge un plateau (parametro k_1)
- **Normalizzazione per lunghezza:** documenti lunghi non avvantaggiati ingiustamente (parametro b)
- Molto veloce (nessuna inferenza neurale)

Punti di forza: preciso su vocabolario tecnico (acronimi, termini specifici)

Limiti: nessuna comprensione semantica (sinonimi = miss), sensibile alla distribuzione del vocabolario

16.4 Hybrid Search: Reciprocal Rank Fusion (RRF)

Problema: FAISS e BM25 hanno punteggi non confrontabili (cosine $\in [0, 1]$ vs score BM25 $\in \mathbb{R}^+$).

Soluzione: combinare le *classifiche* (rankings) anziché i punteggi assoluti.

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

dove R = insieme dei retriever, $k = 60$ (costante di smorzamento standard).

Perché funziona:

- Robusto all'eterogeneità degli score
- Favorisce documenti in cima a più classifiche (complementarietà)
- Nessun tuning manuale di pesi relativi
- Semplice e interpretabile

Procedura Lab 5:

1. BM25: top-20 per score lessicale
2. FAISS: top-20 per cosine similarity semantica
3. RRF: somma $\frac{1}{k + \text{rank}}$ per ogni documento nelle due liste
4. Re-ranking per score RRF finale, restituire top- k

```

1 from rank_bm25 import BM25Okapi
2 import faiss, numpy as np
3
4 # Inizializzazione
5 tokenized_docs = [doc.lower().split() for doc in documents]
6 bm25 = BM25Okapi(tokenized_docs)
7 faiss_index = faiss.IndexFlatIP(768)
8 faiss_index.add(embeddings_normalized)
9
10 def hybrid_retrieve(query, k=5, k_rrf=60):
11     # BM25 retrieval
12     bm25_scores = bm25.get_scores(query.lower().split())
13     bm25_top = np.argsort(bm25_scores)[::-1][:20]
14
15     # FAISS retrieval
16     q_emb = encode_and_normalize(query)
17     _, faiss_top = faiss_index.search(q_emb, 20)
18     faiss_top = faiss_top[0]
19
20     # Reciprocal Rank Fusion
21     rrf_scores = {}
22     for rank, idx in enumerate(bm25_top):
23         rrf_scores[idx] = rrf_scores.get(idx, 0) + 1/(k_rrf + rank + 1)
24     for rank, idx in enumerate(faiss_top):
25         rrf_scores[idx] = rrf_scores.get(idx, 0) + 1/(k_rrf + rank + 1)
26
27     sorted_docs = sorted(rrf_scores.items(), key=lambda x: x[1],
28                          reverse=True)
29     return [documents[i] for i, _ in sorted_docs[:k]]

```

16.5 Componenti del Sistema (Lab 5)

Componente	Implementazione
Dataset	MaartenGr/arxiv_nlp (44.949 abstract NLP)
Encoder	allenai/scibert_scivocab_uncased (768-dim)
Semantic Index	FAISS IndexFlatIP
Lexical Retriever	BM25Okapi (rank-bm25)
Fusion	Reciprocal Rank Fusion ($k = 60$)
LLM Generator	microsoft/Phi-3.5-mini-instruct (3.8B param)
Max tokens gen.	300
Temperature	0.3 (quasi deterministico)

Prompt template:


```

1 system_prompt = (
2     "You are an expert NLP research assistant. "
3     "Answer ONLY based on the provided context. "
4     "Cite the papers you use."
5 )
6 context_text = "\n\n".join([doc[:200] for doc in retrieved_docs
7                               [:5]])
8 context_text = context_text[:3000] # troncamento a 3000 char
9 messages = [
10     {"role": "system", "content": system_prompt},
11     {"role": "user", "content": f"Context:\n{context_text}\n\
12     nQuestion: {query}"}

```

17 Metriche di Valutazione (Lab 5)

17.1 Precision@K

$$P@K = \frac{|\text{documenti rilevanti nei top-}K|}{K}$$

Un documento è “rilevante” se il titolo contiene almeno una keyword del Gold Standard.
Gold Standard Lab 5 – 5 query NLP con keyword attese:

- “BERT / self-attention” → keyword: bert, attention, transformer, self-attention
- “word2vec” → keyword: word2vec, word embeddings, word vectors
- “machine translation” → keyword: translation, multilingual, parallel corpus
- “topic modeling” → keyword: topic, lda, latent dirichlet
- “named entity recognition” → keyword: ner, named entity, entity recognition

17.2 Context Relevance (CR)

$$CR = \frac{1}{|R|} \sum_{d \in R} \cos(\text{emb}(q), \text{emb}(d[:200]))$$

Allineamento semantico medio tra query e snippet dei documenti recuperati.

17.3 Answer Relevance (AR)

$$AR = \cos(\text{emb}(q), \text{emb}(\text{risposta}))$$

Misura se la risposta generata rimane centrata sulla query.

17.4 Faithfulness (FF)

$$\text{FF} = \frac{|\{w \in \text{answer} : w \in \text{context}\}|}{|\text{answer_content_words}|}$$

Proxy lessicale: quante parole di contenuto della risposta compaiono nel contesto recuperato. Alta FF = risposta ancorata ai documenti (meno allucinazioni).

17.5 Robustness

$$\text{Rob} = \frac{1}{|P|} \sum_{p \in P} \frac{|R_5(q) \cap R_5(p)|}{5}$$

Overlap tra risultati per query originale e parafrasi. Alta robustezza = retrieval stabile a riformulazioni.

18 Risultati Lab 5: Esperimento Multi-Corpus

18.1 Disegno Sperimentale

Variare la dimensione del corpus: $N \in \{1.000, 5.000, 15.000, 25.000, 44.949\}$

18.2 Tabella Risultati Completa

N	Metrica	Base (FAISS)	Hybrid (RRF)	Δ	% Migl.
1K	P@5	0.160	0.400	+0.240	+150%
	CR	0.716	0.718	+0.001	trascur.
	AR	0.697	0.684	-0.014	-2%
	FF	0.516	0.588	+0.072	+14%
5K	P@5	0.240	0.560	+0.320	+133%
	CR	0.724	0.714	-0.010	-1.4%
	AR	0.687	0.674	-0.014	-2%
	FF	0.572	0.667	+0.095	+16.6%
15K	P@5	0.160	0.520	+0.360	+225%
	CR	0.713	0.719	+0.007	+1%
	AR	0.686	0.667	-0.018	-2.6%
	FF	0.562	0.647	+0.084	+15%
25K	P@5	0.200	0.440	+0.240	+120%
	CR	0.720	0.730	+0.010	+1.4%
	AR	0.679	0.682	+0.003	+0.4%
	FF	0.560	0.587	+0.027	+4.8%
44.9K	P@5	0.120	0.360	+0.240	+200%
	CR	0.719	0.726	+0.007	+1%
	AR	0.678	0.666	-0.012	-1.8%
	FF	0.496	0.569	+0.074	+14.9%

18.3 Analisi Critica dei Risultati

1. P@5: miglioramento sistematico del Hybrid

L'approccio ibrido migliora consistentemente la P@5 su tutte le dimensioni corpus (+120% a +225%). BM25 recupera paper con acronimi tecnici esatti (NER, LDA, word2vec) che i soli embedding tendono a generalizzare.

2. P@5 non monotona con la dimensione del corpus

- Picco a $N = 5.000$ (P@5 hybrid = 0.560)
- Declino a $N = 44.949$ (P@5 hybrid = 0.360)
- Spiegazione: corpus piccolo = paper rilevanti concentrati, alta densità. Corpus grande = spazio raro, segnale di rilevanza diluito, più candidati competono

3. CR e AR: differenze minime

CR e AR sono costruiti su embedding SciBERT (usato da entrambi i sistemi). BM25 porta documenti lessicalmente precisi ma semanticamente più dispersi → leggero calo AR nel hybrid.

4. Faithfulness: miglioramento consistente

I documenti BM25 contengono i termini esatti della query → il LLM li riusa nell'answer → maggiore overlap lessicale answer-context. Questo è particolarmente utile nei domini tecnici dove il vocabolario è vincolato.

5. Robustness: declino catastrofico con la scala

N docs	Robustness (overlap@5)
1.000	0.233 (23.3%)
5.000	0.100 (10.0%)
15.000	0.033 (3.3%)
25.000	0.000 (0.0%)
44.949	0.033 (3.3%)

In corpus piccoli pochi paper dominanti → le parafrasi recuperano gli stessi. In corpus grandi lo spazio è denso di candidati vicini → una minima riformulazione cambia completamente i top-5. Soluzioni: query expansion, ensemble retriever, re-ranking gerarchico.

6. Precision per query (hybrid)

Query	1K	5K	15K	25K	44.9K
BERT/self-attention	0.20	0.40	0.40	0.00	0.20
word2vec	0.40	0.60	0.60	0.60	0.40
machine translation	0.20	0.40	0.60	0.60	0.40
topic modeling	0.40	0.60	0.20	0.20	0.20
named entity recog.	0.80	0.80	0.80	0.80	0.60
Media	0.40	0.56	0.52	0.44	0.36

“Named entity recognition” è il top performer stabile: acronimo NER molto specifico, BM25 lo identifica bene. “BERT/self-attention” instabile (0.00 a 25K): entrambi i termini sono ubiqui nel corpus NLP.

19 Confronto Approcci: Tabella Sinottica

Aspetto	TF-IDF/Jaccard (Lab 1-2)	BERTopic (Lab 4)	RAG Hybrid (Lab 5)
Task	Misura similarità definizioni; recupero onomasiologico	Scoperta automatica topic	Risposta a domande su corpus
Rappresentazione	Vettori sparsi (BoW / TF-IDF)	Embedding densi + riduzione dim	Embedding densi (FAISS) + sparsi (BM25)
Semantica	Limitata (no sinonimi)	Alta (SentenceTransformer)	Alta semantica + alta precisione lessicale
Scalabilità	Alta, veloce	Media (UMAP/HDBSCAN)	Dipende da corpus (best: 5K-15K)
Output	Score similarità / synset	Lista topic + keywords	Risposta in linguaggio naturale
Limite principale	Manca comprensione semantica profonda	Config errata → failure mode	Robustness degrada con scala

20 Teoria: Embedding e Retrieval

20.1 Dense vs. Sparse Representation

Rappresentazioni sparse (TF-IDF, BM25):

- Dimensione = vocabolario (tipicamente 10^4 – 10^6)
- Quasi tutte le componenti sono zero
- Interpretabili: ogni dimensione = un termine
- Veloci, nessuna inferenza neurale

Rappresentazioni dense (BERT, SentenceTransformer):

- Dimensione fissa ridotta (128–1024)
- Tutte le componenti non-zero
- Catturano significato distribuito, sinonimi, relazioni implicite
- Più lente (require inferenza del modello)

20.2 Sentence Transformers

SentenceTransformer: modello BERT fine-tuned per produrre rappresentazioni a livello di frase.

- Training Siamese/triplet: frasi simili vicine, dissimili lontane nello spazio vettoriale
- Pooling sull'ultimo layer per ottenere vettore singolo per la frase
- Modelli comuni: `all-MiniLM-L6-v2` (384-dim, leggero), `all-mpnet-base-v2` (768-dim, alta qualità)

20.3 SciBERT vs. BERT generale

SciBERT (`allenai/scibert_scivocab_uncased`):

- Stessa architettura BERT (12 layer, 768-dim base)
- Vocabolario costruito su testi scientifici (formule, termini tecnici, notazione matematica)
- Pre-training su 1.14M paper da Semantic Scholar
- Superiore su IR scientifico rispetto a BERT generale

20.4 Cos'è HDBSCAN: Intuizione

HDBSCAN costruisce una gerarchia di cluster basata sulla densità:

1. Calcola la core distance di ogni punto (distanza al k -esimo vicino)
2. Costruisce il minimum spanning tree con distanze di mutua raggiungibilità
3. Condensa la gerarchia: rimuove rami che “si dividono” troppo rapidamente
4. Estrae cluster stabili con Excess of Mass (`eom`)
5. Punti non assegnati a nessun cluster = outlier (-1)

Vantaggio fondamentale per BERTopic: non si specifica quanti topic esistono, emergono dai dati.

21 Riepilogo Formule

Formula	Espressione	Uso
Jaccard (SimLex)	$ A \cap B / A \cup B $	Overlap lessicale definizioni
Cosine Similarity	$\vec{a} \cdot \vec{b} / (\vec{a} \cdot \vec{b})$	SimSem, FAISS, CR, AR
TF-IDF	$\text{tf}(t, d) \times \log(N/\text{df}(t))$	Vettorizzazione Lab1-2
BM25 score	$\sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, D)(k_1+1)}{f(q_i, D) + k_1(1-b+b D /\text{avg})}$	Retrieval lessicale
RRF score	$\sum_r 1/(k + \text{rank}_r(d))$	Fusione ranking ibrido
c-TF-IDF	$\frac{f(t,c)}{ c } \cdot \log \frac{N}{\text{df}(t)}$	Topic keywords (BERTopic)
Precision@K	$ \text{rilevanti} \cap \text{top-}K / K$	Qualità retrieval
Faithfulness	$ \{w \in \text{ans} : w \in \text{ctx}\} / \text{ans} $	Grounding LLM
Robustness	$\frac{1}{ P } \sum_p R_5(q) \cap R_5(p) / 5$	Stabilità a parafrasi
Context Rel.	$\frac{1}{ R } \sum_d \cos(q, d[:200])$	Pertinenza contesto
Answer Rel.	$\cos(\text{emb}(q), \text{emb}(\text{ans}))$	Centramento risposta

22 Domande Tipiche da Esame

1. **Differenza tra SimLex e SimSem:** Jaccard misura overlap lessicale puro su insiemi di parole; TF-IDF coseno misura prossimità semantica in spazio vettoriale. Per concetti astratti SimSem > SimLex perché cattura sinonimi.
2. **Cos'è la ricerca onomasiologica?** Data una definizione, recuperare il synset WordNet corretto. Inverso della consultazione dizionario. Implementata con TF-IDF coseno tra definizione e glosse candidate.
3. **Perché i concetti concreti sono più recuperabili?** Il referente fisico vincola il vocabolario: persone diverse usano parole simili per descriverlo. I concetti astratti hanno alta variabilità e dipendenza culturale.
4. **Descrivere la pipeline BERTopic:** Documenti $\rightarrow$ Embedding (SentenceTransformer) $\rightarrow$ Riduzione dimensionale (UMAP) $\rightarrow$ Clustering (HDBSCAN) $\rightarrow$ Estrazione keyword (c-TF-IDF).
5. **Perché la Config 3 di BERTopic fallisce?** min_cluster_size=150 crea cluster troppo grandi e tematicamente eterogenei. Il c-TF-IDF non riesce a estrarre segnale discriminativo e le stopwords emergono nel top-5.

6. **Cos'è RRF e perché si usa?** Reciprocal Rank Fusion combina ranking da retriever diversi (FAISS + BM25) senza richiedere normalizzazione degli score: $\sum_r 1/(k + \text{rank}_r(d))$. Sfrutta la complementarità di retrieval semantico e lessicale.
7. **Trend P@5 vs. corpus size:** non monotono, picco a 5K. Corpus troppo piccolo = pochi documenti rilevanti. Corpus troppo grande = segnale di rilevanza diluito in spazio denso di candidati.
8. **Perché la robustezza crolla a scala?** In corpus grandi lo spazio embedding è molto denso: una minima riformulazione della query sposta i top-k verso documenti simili ma diversi. Fix: query expansion, ensemble retrieval, re-ranking.
9. **Differenza BM25 vs. TF-IDF:** BM25 aggiunge saturazione della frequenza dei termini (k_1) e normalizzazione per lunghezza del documento (b), migliorando il ranking rispetto al semplice TF-IDF.
10. **Cosa misura la Faithfulness?** La percentuale di parole di contenuto della risposta LLM che compaiono nel contesto recuperato. Proxy lessicale per l'assenza di allucinazioni.